SC324 Machine Learning Final Project

William Cerepak

May 15, 2026

==

Loading Packages

```
# PACKAGES
library(splines)
library(ISLR2)
library(gam)
```

```
## Loading required package: foreach
```

```
## Loaded gam 1.22-7
```

```
library(akima)
library(ggplot2)
library(tree) # For Classification Tree Model
library(randomForest) # For Random Forests
```

```
## randomForest 4.7-1.2
```

```
## Type rfNews() to see new features/changes/bug fixes.
```

```
##
## Attaching package: 'randomForest'
```

```
## The following object is masked from 'package:ggplot2':
##
##     margin
```

```
library(gbm) # For Boosting
```

```
## Loaded gbm 2.2.3
```

```
## This version of gbm is no longer under development. Consider transitioning to gbm3, https://github.com/gbm-dev
elopers/gbm3
```

```
library(e1071)
```

```
##
## Attaching package: 'e1071'
```

```
## The following object is masked from 'package:ggplot2':
##
##     element
```

```
library(ROCR) # For ROC Curves
library(glmnet)
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 5.0
```

```
library(jpeg)
library(Matrix)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following object is masked from 'package:randomForest':
##
##     combine
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(modelsummary)
library(leaps)
library(caret)
```

```
## Loading required package: lattice
```

```
library(keras)
```

```
## The keras package is deprecated. Please use the keras3 package instead.
## Alternatively, to continue using legacy keras, call `py_require_legacy_keras()`.
```

```
library(car)        # For VIF and Residual Plots
```

```
## Loading required package: carData
```

```
##
## Attaching package: 'car'
```

```
## The following object is masked from 'package:dplyr':
##
##     recode
```

```
library(tidyverse)  # For data manipulation and ggplot
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✔ forcats   1.0.1     ✔ stringr   1.6.0
## ✔ lubridate 1.9.5     ✔ tibble    3.3.1
## ✔ purrr     1.2.1     ✔ tidyr     1.3.2
## ✔ readr     2.1.6
```

```
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ purrr::accumulate()    masks foreach::accumulate()
## ✖ dplyr::combine()       masks randomForest::combine()
## ✖ e1071::element()       masks ggplot2::element()
## ✖ tidyr::expand()        masks Matrix::expand()
## ✖ dplyr::filter()        masks stats::filter()
## ✖ dplyr::lag()           masks stats::lag()
## ✖ purrr::lift()          masks caret::lift()
## ✖ randomForest::margin() masks ggplot2::margin()
## ✖ tidyr::pack()          masks Matrix::pack()
## ✖ car::recode()          masks dplyr::recode()
## ✖ purrr::some()          masks car::some()
## ✖ tidyr::unpack()        masks Matrix::unpack()
## ✖ purrr::when()          masks foreach::when()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(GGally)     # For the Scatter Plot Matrix
library(corrplot)   # For the Correlation Heatmap
```

```
## corrplot 0.95 loaded
```

Loading Data

```
# DATA
ProjectData_SP2026 <- read.csv("~/Desktop/ProjectData_SP2026.csv")
forest_data_raw <- ProjectData_SP2026
forest_data <- ProjectData_SP2026
```

Survey Response Variable Chosen: VOL.BF

VOL.BF (Net Board-Foot Volume): A “board foot” is a specialized unit representing a piece of lumber 12” x 12” x 1”. This metric estimates how much usable lumber can actually be sawn out of the tree after accounting for the “kerf” (the sawdust lost to the saw blade) and the round shape of the log.

Regression or Classification Problem?

I consider the scenario of running VOL.BF as a categorical variable in classification problem by the following bins:

VOL.BF = 0 — Non-Sawtimber (Class 0)

0 < VOL.BF ≤ 150 — Small Sawtimber (Class 1)

150 < VOL.BF ≤ 400 — Standard Sawtimber (Class 2)

400 < VOL.BF — Large Sawtimber / Old Growth (Class 3)

However, ultimately I want to run VOL.BF as a quantitative variable and will study a regression problem, leaving VOL.BF as numeric.

Regression Problem for VOL.BF

```
# CHOOSE RESPONSE VOL.BF
# Remove other survey variables from data
forest_data <- forest_data |> select(-BA, -VOL.CF, -COUNT, -DRYBIO, -CARBON)
head(forest_data)
```

```
##   SECTION tcc16 elev eastness northness    tri    tpi  ppt tmean tmax tmin
## 1   M333C    70 1786      -30        95 21.750 -4.250 1656   220  709 -268
## 2   M333A    62 1531       84       -53 22.500  4.750  917   480  961    0
## 3   M333B    38 1511       16       -98 16.125  4.375  943   488  925   50
## 4   M333A    65 1288      -12        99  8.750 -3.000  430   488 1065  -88
## 5   M333C    72 1656      -98       -17 45.750 -4.500 1301   352  825 -121
## 6   M333A    78  725       15        98 31.875  4.125  566   770 1399  141
##   tmin01 vpdmax tdmean def tnt   evi ndvi    VOL.BF train
## 1  -1031    661   -430  43   1 22867 8306 13246.316     1
## 2   -689    810   -103  58   1 20238 7488  8816.547     1
## 3   -707    753    -32 106   1 21271 8102 20798.944     1
## 4   -926    992   -211 385   1 19358 7465  3901.247     1
## 5   -928    718   -270 168   1 22571 7378 11443.879     1
## 6   -690   1292     66 228   1 23258 8167  7667.967     1
```

```
# SPLIT THE DATA
# Split the data
train_data <- subset(forest_data, train == 1)
train_data <- subset(train_data, select = -c(train))
test_data  <- subset(forest_data, train == 0)
test_data <- subset(test_data, select = -c(train))
```

==

Descriptive Statistics

```
# Descriptive Statistics
dim(forest_data)
```

```
## [1] 8572   20
```

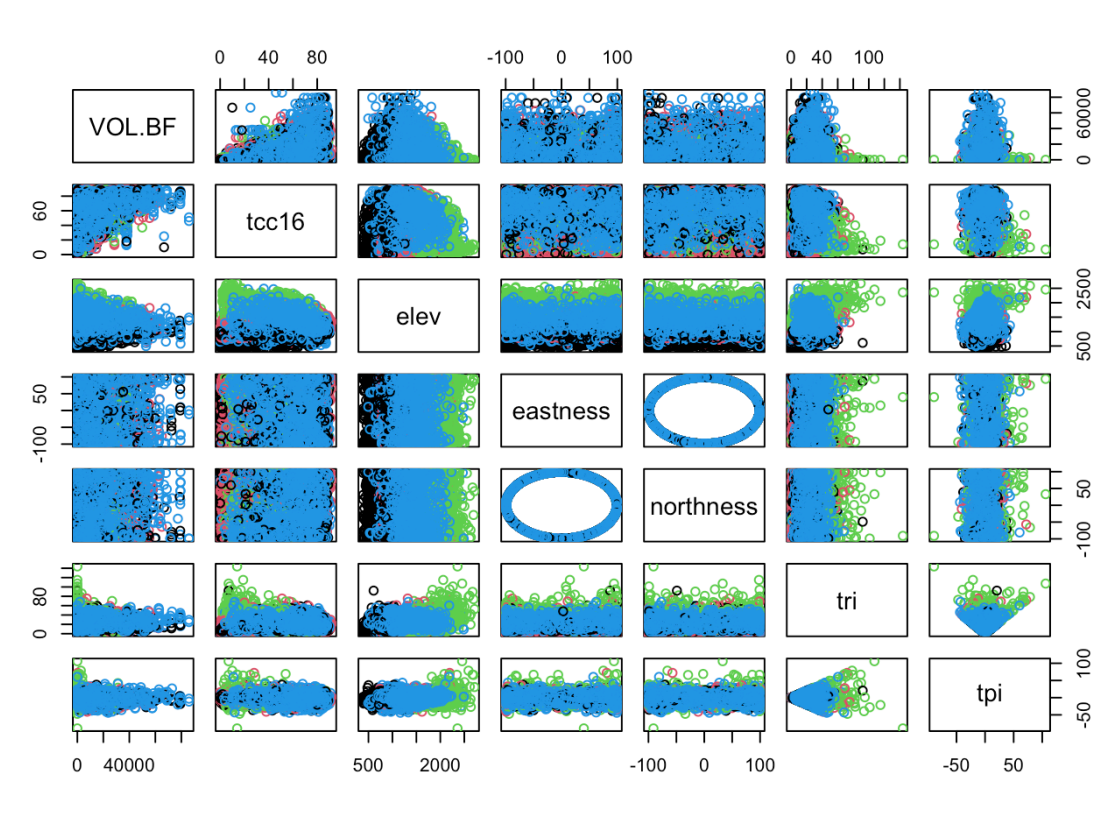
```
summary(forest_data)
```

```
##    SECTION              tcc16            elev         eastness
##  Length:8572        Min.   : 0.00   Min.   : 384   Min.   :-100.000
##  Class :character   1st Qu.:29.00   1st Qu.: 926   1st Qu.: -73.000
##  Mode  :character   Median :51.00   Median :1210   Median :  -1.000
##                     Mean   :48.35   Mean   :1258   Mean   :  -1.387
##                     3rd Qu.:69.00   3rd Qu.:1551   3rd Qu.:  70.000
##                     Max.   :92.00   Max.   :2714   Max.   : 100.000
##    northness            tri               tpi                ppt
##  Min.   :-100.000   Min.   :  0.125   Min.   :-89.37500   Min.   : 256.0
##  1st Qu.: -70.000   1st Qu.: 10.625   1st Qu.: -4.00000   1st Qu.: 578.0
##  Median :  -4.000   Median : 19.125   Median :  0.00000   Median : 817.0
##  Mean   :  -2.431   Mean   : 20.570   Mean   :  0.06088   Mean   : 889.2
##  3rd Qu.:  66.000   3rd Qu.: 29.125   3rd Qu.:  4.25000   3rd Qu.:1145.0
##  Max.   : 100.000   Max.   :143.875   Max.   :106.00000   Max.   :3166.0
##      tmean             tmax           tmin            tmin01
##  Min.   :-178.0   Min.   : 313   Min.   :-669.00   Min.   :-1272.0
##  1st Qu.: 434.0   1st Qu.: 967   1st Qu.:-131.00   1st Qu.: -893.2
##  Median : 585.0   Median :1165   Median :  16.00   Median : -723.0
##  Mean   : 562.3   Mean   :1134   Mean   :  -8.94   Mean   : -744.5
##  3rd Qu.: 713.0   3rd Qu.:1330   3rd Qu.: 122.00   3rd Qu.: -600.0
##  Max.   :1073.0   Max.   :1688   Max.   : 459.00   Max.   : -283.0
##      vpdmax          tdmean              def             tnt
##  Min.   : 447   Min.   :-847.00   Min.   : 34.0   Min.   :0.0000
##  1st Qu.: 864   1st Qu.:-194.00   1st Qu.:269.0   1st Qu.:1.0000
##  Median :1063   Median : -47.00   Median :325.0   Median :1.0000
##  Mean   :1034   Mean   : -81.03   Mean   :340.4   Mean   :0.8098
##  3rd Qu.:1211   3rd Qu.:  66.00   3rd Qu.:395.0   3rd Qu.:1.0000
##  Max.   :1650   Max.   : 308.00   Max.   :789.0   Max.   :1.0000
##       evi              ndvi            VOL.BF            train
##  Min.   :-32768   Min.   :-9890   Min.   :    0.0   Min.   :0.0
##  1st Qu.: 15097   1st Qu.: 6182   1st Qu.:  372.6   1st Qu.:0.0
##  Median : 19497   Median : 7449   Median : 5850.3   Median :0.5
##  Mean   : 17974   Mean   : 6880   Mean   :10314.5   Mean   :0.5
##  3rd Qu.: 22022   3rd Qu.: 8092   3rd Qu.:14970.9   3rd Qu.:1.0
##  Max.   : 32767   Max.   : 9345   Max.   :85885.8   Max.   :1.0
```

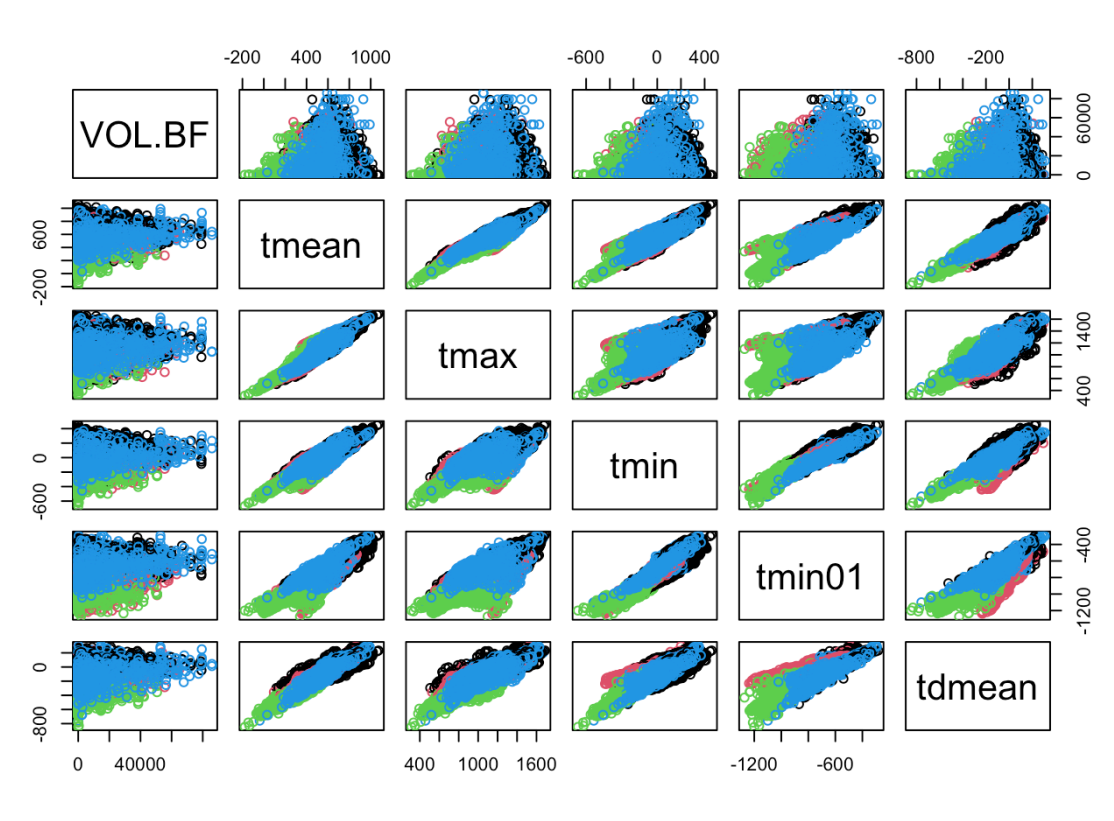
```
glimpse(forest_data)
```

```
## Rows: 8,572
## Columns: 20
## $ SECTION   <chr> "M333C", "M333A", "M333B", "M333A", "M333C", "M333A", "M333A…
## $ tcc16     <int> 70, 62, 38, 65, 72, 78, 46, 28, 69, 34, 42, 69, 39, 66, 3, 2…
## $ elev      <int> 1786, 1531, 1511, 1288, 1656, 725, 666, 1408, 1070, 781, 114…
## $ eastness  <int> -30, 84, 16, -12, -98, 15, -97, 99, -65, 70, -57, -54, 0, 99…
## $ northness <int> 95, -53, -98, 99, -17, 98, -22, 13, 75, -70, 81, 83, 100, -1…
## $ tri       <dbl> 21.750, 22.500, 16.125, 8.750, 45.750, 31.875, 2.500, 12.125…
## $ tpi       <dbl> -4.250, 4.750, 4.375, -3.000, -4.500, 4.125, 0.000, 8.125, -…
## $ ppt       <int> 1656, 917, 943, 430, 1301, 566, 586, 793, 503, 394, 481, 562…
## $ tmean     <int> 220, 480, 488, 488, 352, 770, 809, 515, 592, 722, 512, 731, …
## $ tmax      <int> 709, 961, 925, 1065, 825, 1399, 1360, 1030, 1236, 1397, 1131…
## $ tmin      <int> -268, 0, 50, -88, -121, 141, 257, 0, -51, 46, -107, 110, 265…
## $ tmin01    <int> -1031, -689, -707, -926, -928, -690, -519, -704, -867, -796,…
## $ vpdmax    <int> 661, 810, 753, 992, 718, 1292, 1119, 891, 1251, 1391, 1104, …
## $ tdmean    <int> -430, -103, -32, -211, -270, 66, 251, -80, -77, 67, -217, 49…
## $ def       <int> 43, 58, 106, 385, 168, 228, 221, 219, 247, 558, 366, 266, 33…
## $ tnt       <int> 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, …
## $ evi       <int> 22867, 20238, 21271, 19358, 22571, 23258, 9591, 17402, 22992…
## $ ndvi      <int> 8306, 7488, 8102, 7465, 7378, 8167, 4557, 7189, 8108, 5290, …
## $ VOL.BF    <dbl> 13246.316, 8816.547, 20798.944, 3901.247, 11443.879, 7667.96…
## $ train     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, …
```

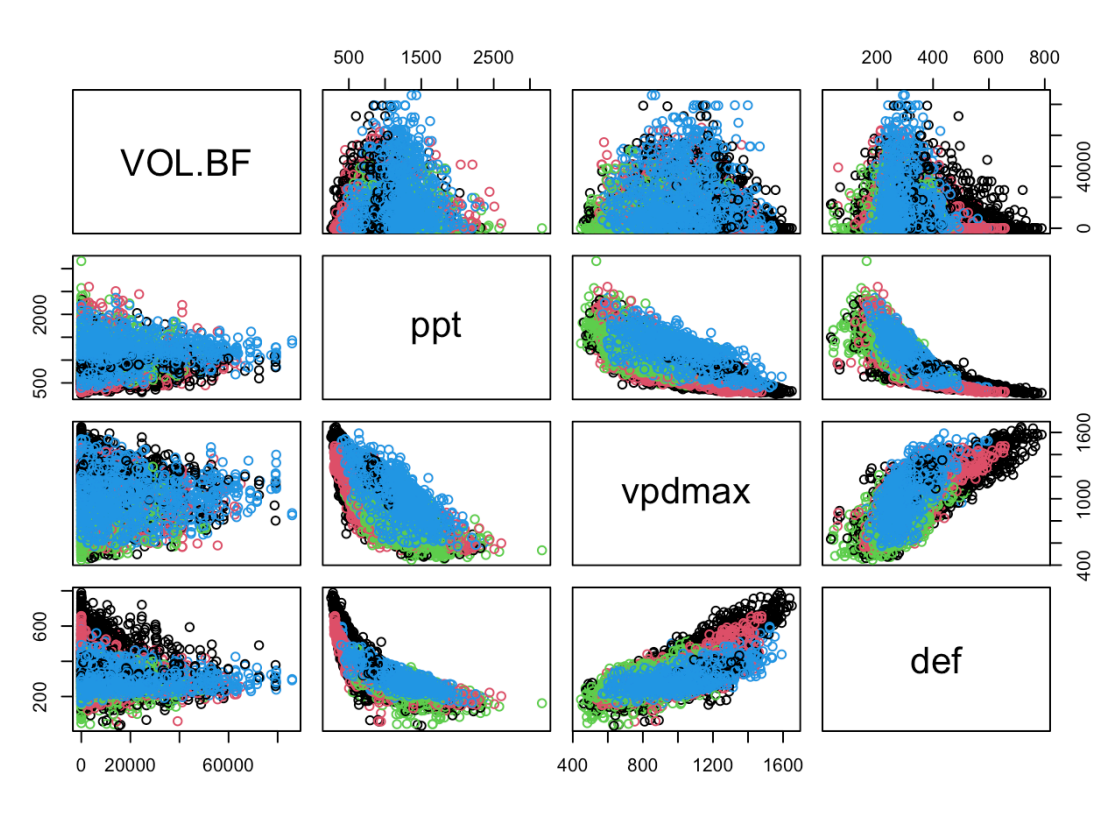
```
# Pairwise Scatterplots
forest_data$section_level <- as.numeric(as.factor(forest_data$SECTION))
forest_data <- forest_data[, -1]

# Plot 1: Topography Variables
pairs(forest_data[, c("VOL.BF", "tcc16", "elev", "eastness", "northness", "tri", "tpi")],
      col = forest_data$section_level)
```



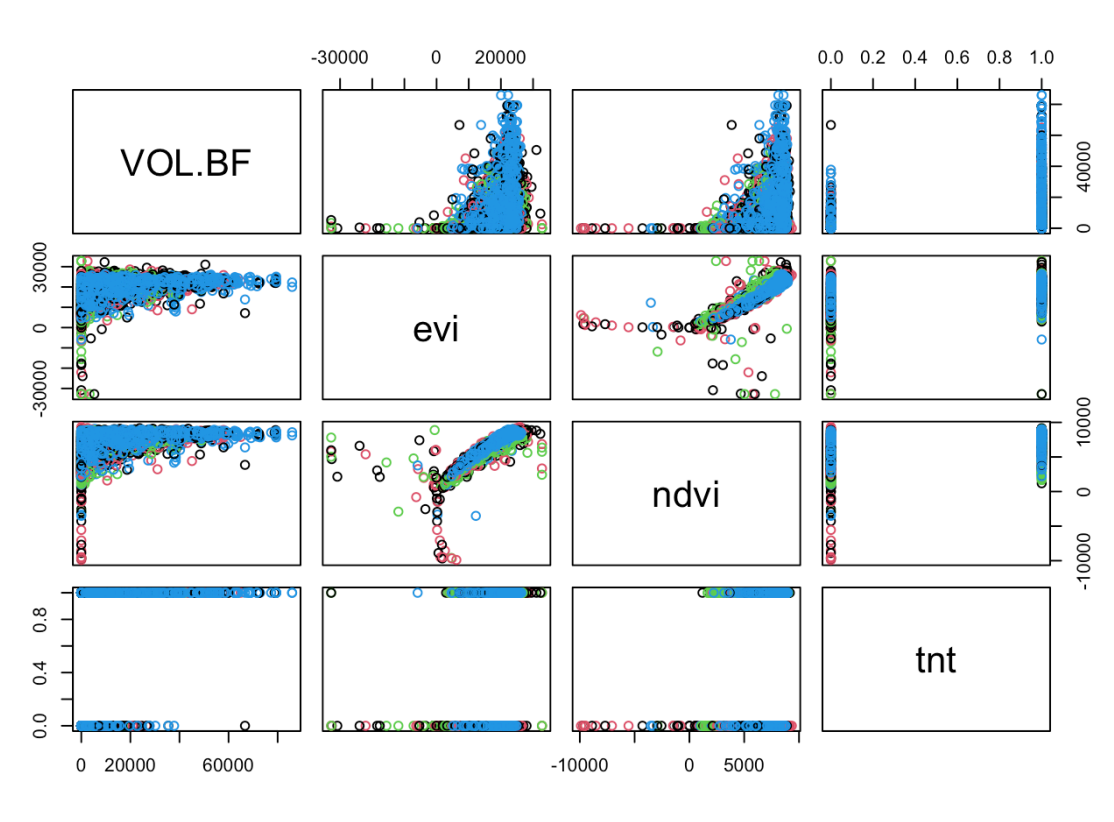
```
# Plot 2: Temperature Variables
pairs(forest_data[, c("VOL.BF", "tmean", "tmax", "tmin", "tmin01", "tdmean")],
      col = forest_data$section_level)
```



```
# Plot 3: Other Climate Variables
pairs(forest_data[, c("VOL.BF", "ppt", "vpdmax", "def")],
      col = forest_data$section_level)
```



```
# Plot 4: LANDFIRE variables
pairs(forest_data[, c("VOL.BF", "evi", "ndvi", "tnt")],
      col = forest_data$section_level)
```



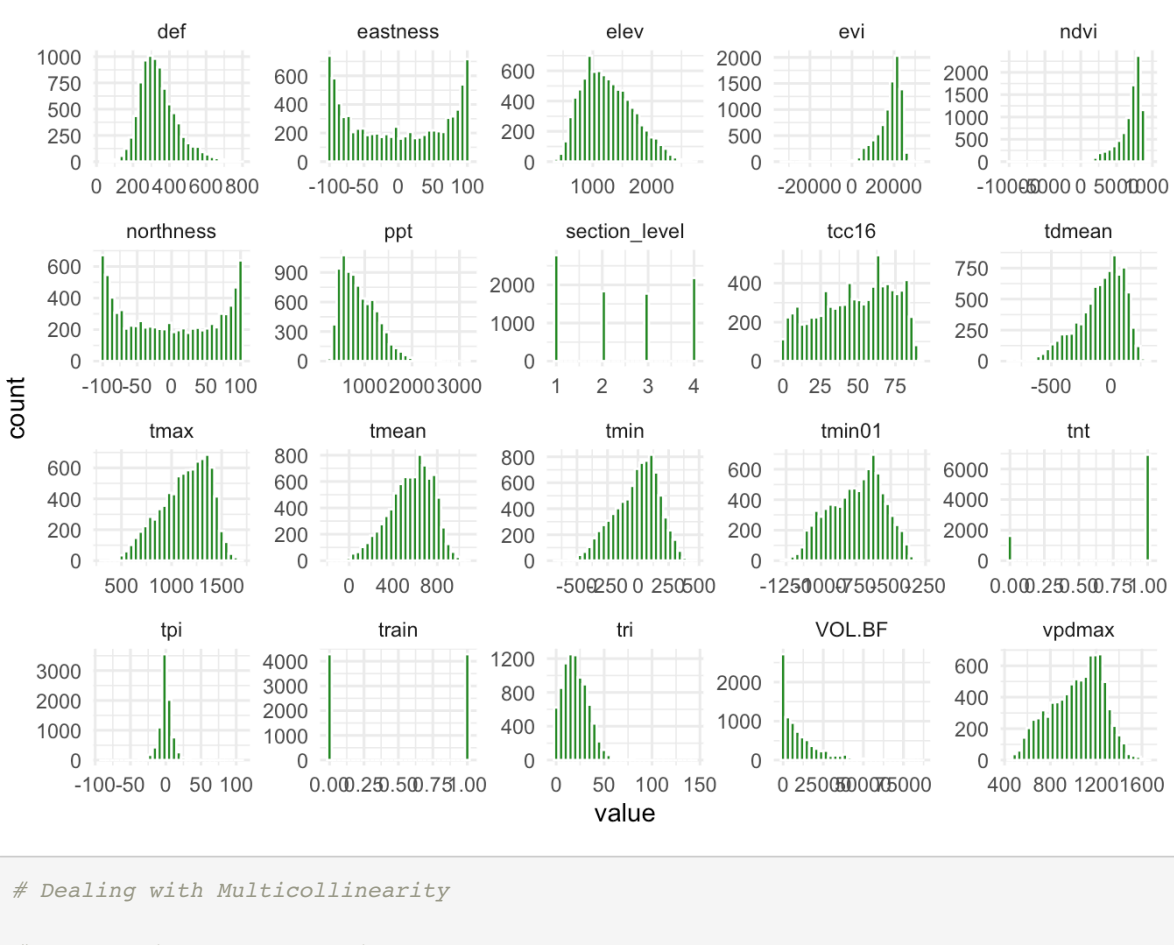
```
# Correlation Matrix
cor(forest_data)
```

```
##                      tcc16         elev     eastness    northness          tri
## tcc16          1.000000000 -0.024156838 -0.015800010  0.1979250259  0.108303480
## elev          -0.024156838  1.000000000  0.034741258  0.0112023157  0.463039710
## eastness      -0.015800010  0.034741258  1.000000000 -0.0158421841  0.009149083
## northness      0.197925026  0.011202316 -0.015842184  1.0000000000  0.041636340
## tri            0.108303480  0.463039710  0.009149083  0.0416363401  1.000000000
## tpi           -0.063436882  0.107801425  0.006838016 -0.0047809767  0.058537475
## ppt            0.167268690  0.650755069 -0.001476625  0.0089037191  0.454446819
## tmean          0.013870892 -0.928956277 -0.025874099 -0.0081485105 -0.395105092
## tmax          -0.009835119 -0.947217128 -0.021528957 -0.0027516579 -0.448394064
## tmin           0.043994453 -0.769167747 -0.028072769 -0.0143218982 -0.265300936
## tmin01         0.088697909 -0.720257011 -0.026739723 -0.0131604065 -0.202625539
## vpdmax        -0.068825610 -0.887221158 -0.017441920  0.0012380771 -0.432306004
## tdmean         0.108623198 -0.904441121 -0.026292629 -0.0063492306 -0.379173780
## def           -0.330037600 -0.609326157 -0.016948763 -0.1386269846 -0.390106182
## tnt            0.620303306  0.152706380  0.001129208  0.1650148581  0.171661014
## evi            0.781576572  0.047175700  0.023676257  0.1719356748  0.096440827
## ndvi           0.767580421  0.035226223  0.021201561  0.1484785554  0.077039393
## VOL.BF         0.568164423 -0.012089810  0.011732960  0.0768583989  0.117244338
## train         -0.002964918  0.002047306 -0.014467803 -0.0001129379  0.017503317
## section_level  0.153586483  0.431136677  0.003427404  0.0224207817  0.344682357
##                        tpi           ppt        tmean         tmax
## tcc16         -0.063436882  1.672687e-01  0.013870892 -0.009835119
## elev           0.107801425  6.507551e-01 -0.928956277 -0.947217128
## eastness       0.006838016 -1.476625e-03 -0.025874099 -0.021528957
## northness     -0.004780977  8.903719e-03 -0.008148511 -0.002751658
## tri            0.058537475  4.544468e-01 -0.395105092 -0.448394064
## tpi            1.000000000  4.633911e-03 -0.038049402 -0.046243500
## ppt            0.004633911  1.000000e+00 -0.618526061 -0.687570926
## tmean         -0.038049402 -6.185261e-01  1.000000000  0.955173156
## tmax          -0.046243500 -6.875709e-01  0.955173156  1.000000000
## tmin          -0.021302305 -4.348476e-01  0.915592079  0.755513051
## tmin01        -0.017663186 -2.724040e-01  0.870776924  0.730308533
## vpdmax        -0.047361022 -7.186961e-01  0.884489067  0.971254521
## tdmean        -0.031774995 -5.093506e-01  0.934644789  0.875078427
## def           -0.022768291 -7.842512e-01  0.650535939  0.678551704
## tnt           -0.012888575  2.202901e-01 -0.156299924 -0.167775966
## evi           -0.107414152  2.565740e-01 -0.087093607 -0.108698445
## ndvi          -0.095409579  2.448997e-01 -0.055065743 -0.084797891
## VOL.BF        -0.076409675  1.851297e-01  0.027362705 -0.006623878
## train          0.001247940 -3.380985e-06 -0.002697080 -0.003725275
## section_level  0.001116372  4.893055e-01 -0.313191040 -0.304749011
##                        tmin       tmin01       vpdmax       tdmean
## tcc16          0.0439944527  0.088697909 -0.068825610  0.108623198
## elev          -0.7691677466 -0.720257011 -0.887221158 -0.904441121
## eastness      -0.0280727695 -0.026739723 -0.017441920 -0.026292629
## northness     -0.0143218982 -0.013160406  0.001238077 -0.006349231
## tri           -0.2653009355 -0.202625539 -0.432306004 -0.379173780
## tpi           -0.0213023050 -0.017663186 -0.047361022 -0.031774995
## ppt           -0.4348476088 -0.272404024 -0.718696110 -0.509350639
## tmean          0.9155920792  0.870776924  0.884489067  0.934644789
## tmax           0.7555130513  0.730308533  0.971254521  0.875078427
## tmin           1.0000000000  0.935080312  0.638137651  0.879729983
## tmin01         0.9350803125  1.000000000  0.605062295  0.871517164
## vpdmax         0.6381376507  0.605062295  1.000000000  0.751013731
## tdmean         0.8797299827  0.871517164  0.751013731  1.000000000
## def            0.5179905379  0.384869946  0.733438243  0.467647206
## tnt           -0.1180779856 -0.076549170 -0.198974806 -0.084665467
## evi           -0.0451340731 -0.011589265 -0.166080596  0.015420708
## ndvi          -0.0067225049  0.033255613 -0.150555470  0.045442784
## VOL.BF         0.0695163165  0.115283561 -0.054957800  0.090318296
## train         -0.0009269393 -0.001395315 -0.005122070 -0.001149997
## section_level -0.2791783652 -0.142027867 -0.273895019 -0.329390712
##                         def          tnt           evi         ndvi
## tcc16         -3.300376e-01  0.620303306  0.7815765724  0.767580421
## elev          -6.093262e-01  0.152706380  0.0471756997  0.035226223
## eastness      -1.694876e-02  0.001129208  0.0236762567  0.021201561
## northness     -1.386270e-01  0.165014858  0.1719356748  0.148478555
## tri           -3.901062e-01  0.171661014  0.0964408271  0.077039393
## tpi           -2.276829e-02 -0.012888575 -0.1074141519 -0.095409579
## ppt           -7.842512e-01  0.220290123  0.2565740201  0.244899690
## tmean          6.505359e-01 -0.156299924 -0.0870936072 -0.055065743
## tmax           6.785517e-01 -0.167775966 -0.1086984450 -0.084797891
## tmin           5.179905e-01 -0.118077986 -0.0451340731 -0.006722505
## tmin01         3.848699e-01 -0.076549170 -0.0115892649  0.033255613
## vpdmax         7.334382e-01 -0.198974806 -0.1660805957 -0.150555470
## tdmean         4.676472e-01 -0.084665467  0.0154207083  0.045442784
## def            1.000000e+00 -0.359820482 -0.4022874215 -0.379862079
## tnt           -3.598205e-01  1.000000000  0.6159382082  0.623724963
## evi           -4.022874e-01  0.615938208  1.0000000000  0.899348748
## ndvi          -3.798621e-01  0.623724963  0.8993487476  1.000000000
## VOL.BF        -2.326874e-01  0.319514249  0.4350671641  0.420379328
## train         -9.533247e-05  0.009512921  0.0007501266  0.002301558
## section_level -3.843340e-01  0.162751673  0.1496629010  0.162514087
##                     VOL.BF         train section_level
## tcc16          0.568164423 -2.964918e-03   0.153586483
## elev          -0.012089810  2.047306e-03   0.431136677
## eastness       0.011732960 -1.446780e-02   0.003427404
## northness      0.076858399 -1.129379e-04   0.022420782
## tri            0.117244338  1.750332e-02   0.344682357
## tpi           -0.076409675  1.247940e-03   0.001116372
## ppt            0.185129737 -3.380985e-06   0.489305488
## tmean          0.027362705 -2.697080e-03  -0.313191040
## tmax          -0.006623878 -3.725275e-03  -0.304749011
## tmin           0.069516317 -9.269393e-04  -0.279178365
## tmin01         0.115283561 -1.395315e-03  -0.142027867
## vpdmax        -0.054957800 -5.122070e-03  -0.273895019
## tdmean         0.090318296 -1.149997e-03  -0.329390712
## def           -0.232687413 -9.533247e-05  -0.384333967
## tnt            0.319514249  9.512921e-03   0.162751673
## evi            0.435067164  7.501266e-04   0.149662901
## ndvi           0.420379328  2.301558e-03   0.162514087
## VOL.BF         1.000000000  3.877592e-03   0.159068742
## train          0.003877592  1.000000e+00   0.001184709
## section_level  0.159068742  1.184709e-03   1.000000000
```

```
# Checking for Sparsity
forest_data |> group_by(section_level) |> count(section_level)
```

```
## # A tibble: 4 × 2
## # Groups:   section_level [4]
##   section_level     n
##           <dbl> <int>
## 1             1  2780
## 2             2  1835
## 3             3  1774
## 4             4  2183
```

```
forest_data |>
  select(where(is.numeric)) |>
  pivot_longer(everything()) |>
  ggplot(aes(x = value)) +
  geom_histogram(bins = 30, fill = "forestgreen", color = "white") +
  facet_wrap(~name, scales = "free") +
  theme_minimal()
```



```
# Dealing with Multicollinearity

# Calculating VIF for Variables
model_full <- lm(VOL.BF ~ ., data = train_data) # Full model with all predictors
vif_values <- vif(model_full) # VIF values
print(vif_values) # Display the results
```

```
##                   GVIF Df GVIF^(1/(2*Df))
## SECTION   6.166362e+00  3        1.354165
## tcc16     3.211324e+00  1        1.792017
## elev      2.060619e+01  1        4.539404
## eastness  1.012510e+00  1        1.006235
## northness 1.107147e+00  1        1.052211
## tri       1.462982e+00  1        1.209538
## tpi       1.119750e+00  1        1.058183
## ppt       4.829828e+00  1        2.197687
## tmean     2.425334e+05  1      492.476771
## tmax      9.102622e+04  1      301.705522
## tmin      4.976584e+04  1      223.082596
## tmin01    1.738425e+01  1        4.169443
## vpdmax    8.261261e+01  1        9.089148
## tdmean    3.038330e+01  1        5.512105
## def       5.162753e+00  1        2.272169
## tnt       1.928755e+00  1        1.388796
## evi       6.979272e+00  1        2.641831
## ndvi      6.580071e+00  1        2.565165
```

```
model_vif <- lm(VOL.BF ~ . -tmean -tmin01 -tmax -tdmean -ndvi -vpdmax, # Remove redundant variables
                data = train_data)
vif_values <- vif(model_vif)
print(vif_values)
```

```
##               GVIF Df GVIF^(1/(2*Df))
## SECTION   3.024948  3        1.202596
## tcc16     3.092319  1        1.758499
## elev      4.171334  1        2.042384
## eastness  1.010478  1        1.005225
## northness 1.072907  1        1.035812
## tri       1.408740  1        1.186904
## tpi       1.058899  1        1.029028
## ppt       3.879069  1        1.969535
## tmin      3.540807  1        1.881703
## def       3.701008  1        1.923800
## tnt       1.871175  1        1.367909
## evi       3.128797  1        1.768841
```

==

Linear Regression Models

```
# Simple baseline intercept-only model
model_simple <- lm(VOL.BF ~ 1, data = train_data)
model_simple
```

```
##
## Call:
## lm(formula = VOL.BF ~ 1, data = train_data)
##
## Coefficients:
## (Intercept)
##       10364
```

```
# Get predictions
simple_preds <- predict(model_simple, newdata = test_data)

test_mse <- mean((test_data$VOL.BF - simple_preds)^2) # Test MSE
test_rmse <- sqrt(test_mse)
print(paste("Test MSE:", round(test_mse, 2)))
```

```
## [1] "Test MSE: 163898034.07"
```

```
print(paste("Ridge Test RMSE:", round(test_rmse, 2)))
```

```
## [1] "Ridge Test RMSE: 12802.27"
```

MSE = 163898034

```
# Full model
model_full <- lm(VOL.BF ~ ., data = train_data)
summary(model_full)
```

```
##
## Call:
## lm(formula = VOL.BF ~ ., data = train_data)
##
## Residuals:
##    Min     1Q Median     3Q    Max
## -25970  -6227  -1345   3432  66038
##
## Coefficients:
##                 Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -1.859e+04  7.065e+03  -2.631  0.00855 **
## SECTIONM333B   8.498e+02  5.080e+02   1.673  0.09444 .
## SECTIONM333C  -4.145e+02  6.964e+02  -0.595  0.55172
## SECTIONM333D   1.552e+03  5.598e+02   2.772  0.00560 **
## tcc16          3.125e+02  1.155e+01  27.064  < 2e-16 ***
## elev           1.637e+00  1.689e+00   0.969  0.33238
## eastness       4.959e+00  2.221e+00   2.233  0.02561 *
## northness     -6.078e+00  2.395e+00  -2.538  0.01119 *
## tri            4.557e+01  1.437e+01   3.171  0.00153 **
## tpi           -6.387e+01  1.704e+01  -3.749  0.00018 ***
## ppt            4.410e+00  9.175e-01   4.806 1.59e-06 ***
## tmean         -1.194e+02  3.913e+02  -0.305  0.76029
## tmax           7.367e+01  1.956e+02   0.377  0.70645
## tmin           6.720e+01  1.957e+02   0.343  0.73139
## tmin01        -4.856e+00  3.412e+00  -1.423  0.15481
## vpdmax        -7.415e+00  6.300e+00  -1.177  0.23923
## tdmean        -1.089e+00  4.567e+00  -0.238  0.81158
## def           -2.514e+00  3.502e+00  -0.718  0.47287
## tnt           -1.584e+03  5.609e+02  -2.825  0.00476 **
## evi            3.007e-02  7.518e-02   0.400  0.68918
## ndvi          -5.474e-01  2.356e-01  -2.323  0.02020 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 10300 on 4265 degrees of freedom
## Multiple R-squared:  0.3507, Adjusted R-squared:  0.3476
## F-statistic: 115.2 on 20 and 4265 DF,  p-value: < 2.2e-16
```

```
# Get predictions
full_preds <- predict(model_full, newdata = test_data)

mean((test_data$VOL.BF - full_preds)^2) # Test MSE
```

```
## [1] 106279231
```

MSE = 106279231

```
# Backward Selection
model_bwd <- step(model_full, direction = "backward")
```

```
## Start:  AIC=79223.35
## VOL.BF ~ SECTION + tcc16 + elev + eastness + northness + tri +
##     tpi + ppt + tmean + tmax + tmin + tmin01 + vpdmax + tdmean +
##     def + tnt + evi + ndvi
##
##             Df  Sum of Sq        RSS   AIC
## - tdmean     1 6.0268e+06 4.5227e+11 79221
## - tmean      1 9.8717e+06 4.5227e+11 79221
## - tmin       1 1.2497e+07 4.5227e+11 79221
## - tmax       1 1.5044e+07 4.5228e+11 79221
## - evi        1 1.6966e+07 4.5228e+11 79222
## - def        1 5.4649e+07 4.5232e+11 79222
## - elev       1 9.9660e+07 4.5236e+11 79222
## - vpdmax     1 1.4692e+08 4.5241e+11 79223
## <none>                    4.5226e+11 79223
## - tmin01     1 2.1472e+08 4.5248e+11 79223
## - eastness   1 5.2871e+08 4.5279e+11 79226
## - ndvi       1 5.7246e+08 4.5283e+11 79227
## - northness  1 6.8301e+08 4.5294e+11 79228
## - tnt        1 8.4604e+08 4.5311e+11 79229
## - SECTION    3 1.4551e+09 4.5372e+11 79231
## - tri        1 1.0665e+09 4.5333e+11 79231
## - tpi        1 1.4906e+09 4.5375e+11 79235
## - ppt        1 2.4496e+09 4.5471e+11 79244
## - tcc16      1 7.7672e+10 5.2993e+11 79901
##
## Step:  AIC=79221.4
## VOL.BF ~ SECTION + tcc16 + elev + eastness + northness + tri +
##     tpi + ppt + tmean + tmax + tmin + tmin01 + vpdmax + def +
##     tnt + evi + ndvi
##
##             Df  Sum of Sq        RSS   AIC
## - tmean      1 1.0243e+07 4.5228e+11 79220
## - tmin       1 1.2877e+07 4.5228e+11 79220
## - tmax       1 1.5002e+07 4.5228e+11 79220
## - evi        1 1.6893e+07 4.5228e+11 79220
## - def        1 5.0602e+07 4.5232e+11 79220
## - elev       1 1.3010e+08 4.5240e+11 79221
## - vpdmax     1 1.8127e+08 4.5245e+11 79221
## <none>                    4.5227e+11 79221
## - tmin01     1 2.4007e+08 4.5251e+11 79222
## - eastness   1 5.2977e+08 4.5280e+11 79224
## - ndvi       1 5.6919e+08 4.5284e+11 79225
## - northness  1 6.8099e+08 4.5295e+11 79226
## - tnt        1 8.4570e+08 4.5311e+11 79227
## - tri        1 1.0669e+09 4.5333e+11 79230
## - SECTION    3 1.5674e+09 4.5383e+11 79230
## - tpi        1 1.5192e+09 4.5379e+11 79234
## - ppt        1 2.4744e+09 4.5474e+11 79243
## - tcc16      1 7.7746e+10 5.3001e+11 79899
## 
## Step:  AIC=79219.5
## VOL.BF ~ SECTION + tcc16 + elev + eastness + northness + tri + 
##     tpi + ppt + tmax + tmin + tmin01 + vpdmax + def + tnt + evi + 
##     ndvi
## 
##              Df  Sum of Sq        RSS   AIC
## - evi        1 1.6565e+07 4.5229e+11 79218
## - def        1 4.9661e+07 4.5233e+11 79218
## - elev       1 1.3050e+08 4.5241e+11 79219
## - vpdmax     1 1.8203e+08 4.5246e+11 79219
## <none>                    4.5228e+11 79220
## - tmin01     1 2.3894e+08 4.5252e+11 79220
## - tmax       1 4.1466e+08 4.5269e+11 79221
## - eastness   1 5.2743e+08 4.5280e+11 79222
## - tmin       1 5.4141e+08 4.5282e+11 79223
## - ndvi       1 5.6748e+08 4.5284e+11 79223
## - northness  1 6.7955e+08 4.5296e+11 79224
## - tnt        1 8.4907e+08 4.5313e+11 79226
## - tri        1 1.0676e+09 4.5334e+11 79228
## - SECTION    3 1.5615e+09 4.5384e+11 79228
## - tpi        1 1.5156e+09 4.5379e+11 79232
## - ppt        1 2.4801e+09 4.5476e+11 79241
## - tcc16      1 7.7759e+10 5.3004e+11 79897
## 
## Step:  AIC=79217.66
## VOL.BF ~ SECTION + tcc16 + elev + eastness + northness + tri + 
##     tpi + ppt + tmax + tmin + tmin01 + vpdmax + def + tnt + ndvi
## 
##              Df  Sum of Sq        RSS   AIC
## - def        1 5.3556e+07 4.5235e+11 79216
## - elev       1 1.2119e+08 4.5241e+11 79217
## - vpdmax     1 1.7706e+08 4.5247e+11 79217
## <none>                    4.5229e+11 79218
## - tmin01     1 2.5216e+08 4.5255e+11 79218
## - tmax       1 4.0554e+08 4.5270e+11 79219
## - eastness   1 5.3612e+08 4.5283e+11 79221
## - tmin       1 5.5335e+08 4.5285e+11 79221
## - northness  1 6.7392e+08 4.5297e+11 79222
## - tnt        1 8.3709e+08 4.5313e+11 79224
## - ndvi       1 9.3244e+08 4.5323e+11 79224
## - tri        1 1.0762e+09 4.5337e+11 79226
## - SECTION    3 1.5620e+09 4.5386e+11 79226
## - tpi        1 1.5294e+09 4.5382e+11 79230
## - ppt        1 2.5089e+09 4.5480e+11 79239
## - tcc16      1 8.5532e+10 5.3783e+11 79958
## 
## Step:  AIC=79216.16
## VOL.BF ~ SECTION + tcc16 + elev + eastness + northness + tri + 
##     tpi + ppt + tmax + tmin + tmin01 + vpdmax + tnt + ndvi
## 
##              Df  Sum of Sq        RSS   AIC
## - elev       1 1.0605e+08 4.5245e+11 79215
## <none>                    4.5235e+11 79216
## - tmin01     1 2.4339e+08 4.5259e+11 79216
## - vpdmax     1 3.3129e+08 4.5268e+11 79217
## - tmin       1 4.9986e+08 4.5285e+11 79219
## - tmax       1 5.1915e+08 4.5287e+11 79219
## - eastness   1 5.4793e+08 4.5290e+11 79219
## - northness  1 6.2357e+08 4.5297e+11 79220
## - tnt        1 8.1371e+08 4.5316e+11 79222
## - ndvi       1 9.0986e+08 4.5326e+11 79223
## - tri        1 1.1159e+09 4.5346e+11 79225
## - SECTION    3 1.6124e+09 4.5396e+11 79225
## - tpi        1 1.4964e+09 4.5384e+11 79228
## - ppt        1 3.5680e+09 4.5592e+11 79248
## - tcc16      1 8.7612e+10 5.3996e+11 79973
## 
## Step:  AIC=79215.17
## VOL.BF ~ SECTION + tcc16 + eastness + northness + tri + tpi + 
##     ppt + tmax + tmin + tmin01 + vpdmax + tnt + ndvi
## 
##              Df  Sum of Sq        RSS   AIC
## - tmin01     1 1.8744e+08 4.5264e+11 79215
## <none>                    4.5245e+11 79215
## - vpdmax     1 2.4177e+08 4.5270e+11 79215
## - tmin       1 4.3713e+08 4.5289e+11 79217
## - tmax       1 4.5483e+08 4.5291e+11 79217
## - eastness   1 5.7213e+08 4.5303e+11 79219
## - northness  1 6.0734e+08 4.5306e+11 79219
## - tnt        1 7.8152e+08 4.5323e+11 79221
## - ndvi       1 9.3966e+08 4.5339e+11 79222
## - tri        1 1.1207e+09 4.5357e+11 79224
## - tpi        1 1.3904e+09 4.5384e+11 79226
## - SECTION    3 2.0125e+09 4.5447e+11 79228
## - ppt        1 3.4623e+09 4.5592e+11 79246
## - tcc16      1 8.7507e+10 5.3996e+11 79971
## 
## Step:  AIC=79214.94
## VOL.BF ~ SECTION + tcc16 + eastness + northness + tri + tpi + 
##     ppt + tmax + tmin + vpdmax + tnt + ndvi
## 
##              Df  Sum of Sq        RSS   AIC
## - vpdmax     1 1.6355e+08 4.5280e+11 79214
## <none>                    4.5264e+11 79215
## - tmin       1 2.7228e+08 4.5291e+11 79216
## - tmax       1 3.1032e+08 4.5295e+11 79216
## - eastness   1 5.6290e+08 4.5320e+11 79218
## - northness  1 5.9336e+08 4.5323e+11 79219
## - tnt        1 8.3432e+08 4.5348e+11 79221
## - ndvi       1 8.4218e+08 4.5348e+11 79221
## - tri        1 1.0893e+09 4.5373e+11 79223
## - SECTION    3 1.8266e+09 4.5447e+11 79226
## - tpi        1 1.4196e+09 4.5406e+11 79226
## - ppt        1 3.4780e+09 4.5612e+11 79246
## - tcc16      1 8.7393e+10 5.4003e+11 79970
## 
## Step:  AIC=79214.49
## VOL.BF ~ SECTION + tcc16 + eastness + northness + tri + tpi + 
##     ppt + tmax + tmin + tnt + ndvi
## 
##              Df  Sum of Sq        RSS   AIC
## <none>                    4.5280e+11 79214
## - tmax       1 3.1985e+08 4.5312e+11 79216
## - eastness   1 5.5902e+08 4.5336e+11 79218
## - northness  1 6.2918e+08 4.5343e+11 79218
## - tmin       1 7.5521e+08 4.5356e+11 79220
## - ndvi       1 7.6733e+08 4.5357e+11 79220
## - tnt        1 8.2853e+08 4.5363e+11 79220
## - tri        1 9.9841e+08 4.5380e+11 79222
## - SECTION    3 1.7662e+09 4.5457e+11 79225
## - tpi        1 1.3858e+09 4.5419e+11 79226
## - ppt        1 4.2013e+09 4.5701e+11 79252
## - tcc16      1 8.8168e+10 5.4097e+11 79975
```

```
summary(model_bwd)
```

```
## 
## Call:
## lm(formula = VOL.BF ~ SECTION + tcc16 + eastness + northness + 
##     tri + tpi + ppt + tmax + tmin + tnt + ndvi, data = train_data)
## 
## Residuals:
##    Min     1Q Median     3Q    Max 
## -26258  -6254  -1339   3374  65964 
## 
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)    
## (Intercept)  -8598.9209  2108.4530  -4.078 4.62e-05 ***
## SECTIONM333B   986.8015   478.8776   2.061 0.039396 *  
## SECTIONM333C  -104.7476   624.7802  -0.168 0.866863    
## SECTIONM333D  1560.7638   479.8924   3.252 0.001154 ** 
## tcc16          314.9160    10.9189  28.841  < 2e-16 ***
## eastness         5.0846     2.2140   2.297 0.021693 *  
## northness       -5.6814     2.3319  -2.436 0.014875 *  
## tri             43.5509    14.1900   3.069 0.002160 ** 
## tpi            -58.8362    16.2720  -3.616 0.000303 ***
## ppt              4.3653     0.6934   6.296 3.37e-10 ***
## tmax             2.3038     1.3262   1.737 0.082434 .  
## tmin             4.2220     1.5817   2.669 0.007630 ** 
## tnt          -1555.0377   556.1932  -2.796 0.005199 ** 
## ndvi            -0.4240     0.1576  -2.691 0.007160 ** 
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## 
## Residual standard error: 10300 on 4272 degrees of freedom
## Multiple R-squared:  0.3499, Adjusted R-squared:  0.3479 
## F-statistic: 176.9 on 13 and 4272 DF,  p-value: < 2.2e-16
```

```
# Get predictions
bwd_preds <- predict(model_bwd, newdata = test_data)

mean((test_data$VOL.BF - bwd_preds)^2) # Test MSE
```

```
## [1] 106484408
```

MSE = 106484408

```
# Reduced model via forward selection
model_fwd <- step(model_simple,
                  scope = list(lower = model_simple, upper = model_full),
                  direction = "forward")
```

```
## Start:  AIC=81034.16
## VOL.BF ~ 1
## 
##              Df  Sum of Sq        RSS   AIC
## + tcc16      1 2.2481e+11 4.7170e+11 79366
## + evi        1 1.3285e+11 5.6367e+11 80129
## + ndvi       1 1.2550e+11 5.7101e+11 80185
## + tnt        1 6.9210e+10 6.2730e+11 80588
## + SECTION    3 3.9412e+10 6.5710e+11 80791
## + def        1 3.7349e+10 6.5917e+11 80800
## + ppt        1 2.3261e+10 6.7325e+11 80891
## + tri        1 1.1766e+10 6.8475e+11 80963
## + tmin01     1 8.5958e+09 6.8792e+11 80983
## + tdmean     1 5.2830e+09 6.9123e+11 81004
## + tpi        1 4.6390e+09 6.9187e+11 81008
## + northness  1 3.7076e+09 6.9281e+11 81013
## + tmin       1 3.0126e+09 6.9350e+11 81018
## + vpdmax     1 1.9313e+09 6.9458e+11 81024
## + tmean      1 4.7151e+08 6.9604e+11 81033
## <none>                    6.9651e+11 81034
## + eastness   1 1.5802e+08 6.9636e+11 81035
## + elev       1 8.5609e+07 6.9643e+11 81036
## + tmax       1 2.6503e+07 6.9649e+11 81036
## 
## Step:  AIC=79365.72
## VOL.BF ~ tcc16
## 
##              Df  Sum of Sq        RSS   AIC
## + SECTION    3 7299495156 4.6440e+11 79305
## + ppt        1 4891597834 4.6681e+11 79323
## + tri        1 2821946223 4.6888e+11 79342
## + tmin01     1 2718096676 4.6898e+11 79343
## + tnt        1 1455136557 4.7025e+11 79354
## + tmin       1 1320655447 4.7038e+11 79356
## + tpi        1 1171572425 4.7053e+11 79357
## + def        1 1168040218 4.7053e+11 79357
## + northness  1  998235976 4.7070e+11 79359
## + tdmean     1  569255698 4.7113e+11 79363
## + ndvi       1  536667973 4.7116e+11 79363
## + eastness   1  311147742 4.7139e+11 79365
## + tmean      1  290817820 4.7141e+11 79365
## + evi        1  253280652 4.7145e+11 79365
## <none>                    4.7170e+11 79366
## + vpdmax     1   96368113 4.7160e+11 79367
## + elev       1    1293968 4.7170e+11 79368
## + tmax       1    1004109 4.7170e+11 79368
## 
## Step:  AIC=79304.88
## VOL.BF ~ tcc16 + SECTION
## 
##              Df  Sum of Sq        RSS   AIC
## + ppt        1 3481746038 4.6092e+11 79275
## + tri        1 1714353334 4.6269e+11 79291
## + tnt        1 1331632474 4.6307e+11 79295
## + tpi        1 1326786193 4.6307e+11 79295
## + def        1 1068990227 4.6333e+11 79297
## + northness  1  898698281 4.6350e+11 79299
## + ndvi       1  743942756 4.6366e+11 79300
## + tmin01     1  735957129 4.6367e+11 79300
## + vpdmax     1  482010069 4.6392e+11 79302
## + tmin       1  448731454 4.6395e+11 79303
## + eastness   1  353019405 4.6405e+11 79304
## <none>                    4.6440e+11 79305
## + evi        1  210147623 4.6419e+11 79305
## + tmax       1  177009350 4.6422e+11 79305
## + tdmean     1  103803923 4.6430e+11 79306
## + elev       1   23561049 4.6438e+11 79307
## + tmean      1      38507 4.6440e+11 79307
## 
## Step:  AIC=79274.63
## VOL.BF ~ tcc16 + SECTION + ppt
## 
##              Df  Sum of Sq        RSS   AIC
## + tmean      1 2185209846 4.5873e+11 79256
## + tmin       1 2138796510 4.5878e+11 79257
## + tnt        1 2027490308 4.5889e+11 79258
## + tmin01     1 1668641541 4.5925e+11 79261
## + tdmean     1 1631482353 4.5929e+11 79261
## + ndvi       1 1448367936 4.5947e+11 79263
## + tpi        1 1397436386 4.5952e+11 79264
## + elev       1 1340642851 4.5958e+11 79264
## + tmax       1 1335172969 4.5958e+11 79264
## + vpdmax     1  785042591 4.6013e+11 79269
## + northness  1  756374953 4.6016e+11 79270
## + evi        1  714721666 4.6020e+11 79270
## + tri        1  553754649 4.6037e+11 79271
## + eastness   1  364638310 4.6055e+11 79273
## + def        1  281898665 4.6064e+11 79274
## <none>                    4.6092e+11 79275
## 
## Step:  AIC=79256.26
## VOL.BF ~ tcc16 + SECTION + ppt + tmean
## 
##              Df  Sum of Sq        RSS   AIC
## + tnt        1 1536528748 4.5720e+11 79244
## + ndvi       1 1374772787 4.5736e+11 79245
## + tpi        1 1242028371 4.5749e+11 79247
## + tri        1 1080616518 4.5765e+11 79248
## + northness  1  710135891 4.5802e+11 79252
## + evi        1  606723926 4.5813e+11 79253
## + eastness   1  435798340 4.5830e+11 79254
## <none>                    4.5873e+11 79256
## + vpdmax     1  165183339 4.5857e+11 79257
## + tmin       1  160456415 4.5857e+11 79257
## + tmax       1  159574544 4.5857e+11 79257
## + elev       1   25308504 4.5871e+11 79258
## + def        1    2365095 4.5873e+11 79258
## + tdmean     1     916312 4.5873e+11 79258
## + tmin01     1     802876 4.5873e+11 79258
## 
## Step:  AIC=79243.88
## VOL.BF ~ tcc16 + SECTION + ppt + tmean + tnt
## 
##              Df  Sum of Sq        RSS   AIC
## + tri        1 1245461976 4.5595e+11 79234
## + tpi        1 1127263779 4.5607e+11 79235
## + ndvi       1  731177117 4.5647e+11 79239
## + northness  1  597722233 4.5660e+11 79240
## + eastness   1  469594773 4.5673e+11 79241
## + evi        1  233889164 4.5696e+11 79244
## <none>                    4.5720e+11 79244
## + vpdmax     1  190099193 4.5701e+11 79244
## + tmin       1  169935165 4.5703e+11 79244
## + tmax       1  169267180 4.5703e+11 79244
## + elev       1   47844471 4.5715e+11 79245
## + def        1    5766880 4.5719e+11 79246
## + tmin01     1    4644451 4.5719e+11 79246
## + tdmean     1     152855 4.5720e+11 79246
## 
## Step:  AIC=79234.19
## VOL.BF ~ tcc16 + SECTION + ppt + tmean + tnt + tri
## 
##              Df  Sum of Sq        RSS   AIC
## + tpi        1 1205063415 4.5475e+11 79225
## + northness  1  669546911 4.5528e+11 79230
## + ndvi       1  505476011 4.5545e+11 79231
## + eastness   1  474727626 4.5548e+11 79232
## <none>                    4.5595e+11 79234
## + evi        1  150075181 4.5580e+11 79235
## + vpdmax     1  115034406 4.5584e+11 79235
## + tmin       1   50206104 4.5590e+11 79236
## + tmax       1   49852134 4.5590e+11 79236
## + tdmean     1   10677259 4.5594e+11 79236
## + def        1    8687874 4.5594e+11 79236
## + tmin01     1    4421566 4.5595e+11 79236
## + elev       1    1448394 4.5595e+11 79236
## 
## Step:  AIC=79224.84
## VOL.BF ~ tcc16 + SECTION + ppt + tmean + tnt + tri + tpi
## 
##              Df Sum of Sq        RSS   AIC
## + ndvi       1 658953162 4.5409e+11 79221
## + northness  1 635771290 4.5411e+11 79221
## + eastness   1 500364012 4.5425e+11 79222
## + evi        1 266742538 4.5448e+11 79224
## <none>                   4.5475e+11 79225
## + vpdmax     1 153545973 4.5459e+11 79225
## + elev       1  88799409 4.5466e+11 79226
## + tmin       1  72379461 4.5467e+11 79226
## + tmax       1  71814802 4.5468e+11 79226
## + def        1  21732290 4.5473e+11 79227
## + tdmean     1  16848596 4.5473e+11 79227
## + tmin01     1       149 4.5475e+11 79227
## 
## Step:  AIC=79220.63
## VOL.BF ~ tcc16 + SECTION + ppt + tmean + tnt + tri + tpi + ndvi
## 
##              Df Sum of Sq        RSS   AIC
## + northness  1 657917527 4.5343e+11 79216
## + eastness   1 576358794 4.5351e+11 79217
## <none>                   4.5409e+11 79221
## + vpdmax     1 207125447 4.5388e+11 79221
## + tmin       1  84359545 4.5400e+11 79222
## + tmax       1  83758000 4.5400e+11 79222
## + elev       1  58416246 4.5403e+11 79222
## + def        1  55732981 4.5403e+11 79222
## + tdmean     1  32882572 4.5406e+11 79222
## + evi        1  15419066 4.5407e+11 79222
## + tmin01     1   3522990 4.5408e+11 79223
## 
## Step:  AIC=79216.41
## VOL.BF ~ tcc16 + SECTION + ppt + tmean + tnt + tri + tpi + ndvi + 
##     northness
## 
##             Df Sum of Sq        RSS   AIC
## + eastness  1 564595301 4.5287e+11 79213
## <none>                  4.5343e+11 79216
## + vpdmax    1 165585015 4.5326e+11 79217
## + def       1 137286460 4.5329e+11 79217
## + tmin      1  66361762 4.5336e+11 79218
## + tmax      1  65806002 4.5336e+11 79218
## + elev      1  62448267 4.5337e+11 79218
## + evi       1  25461099 4.5340e+11 79218
## + tdmean    1  22130536 4.5341e+11 79218
## + tmin01    1  10147563 4.5342e+11 79218
## 
## Step:  AIC=79213.07
## VOL.BF ~ tcc16 + SECTION + ppt + tmean + tnt + tri + tpi + ndvi + 
##     northness + eastness
## 
##           Df Sum of Sq        RSS   AIC
## <none>                4.5287e+11 79213
## + vpdmax  1 159031095 4.5271e+11 79214
## + def     1 125967466 4.5274e+11 79214
## + tmin    1  60662627 4.5280e+11 79214
## + tmax    1  60046943 4.5281e+11 79215
## + elev    1  45775865 4.5282e+11 79215
## + tdmean  1  25996679 4.5284e+11 79215
## + evi     1  17923465 4.5285e+11 79215
## + tmin01  1  13236208 4.5285e+11 79215
```

```
summary(model_fwd)
```

```
## 
## Call:
## lm(formula = VOL.BF ~ tcc16 + SECTION + ppt + tmean + tnt + tri + 
##     tpi + ndvi + northness + eastness, data = train_data)
## 
## Residuals:
##    Min     1Q Median     3Q    Max 
## -26299  -6272  -1352   3387  65929 
## 
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)    
## (Intercept)  -9714.3655  1503.4348  -6.461 1.15e-10 ***
## tcc16          314.6157    10.9111  28.835  < 2e-16 ***
## SECTIONM333B   937.7851   474.4716   1.976 0.048164 *  
## SECTIONM333C  -233.4472   601.1354  -0.388 0.697781    
## SECTIONM333D  1497.8090   472.5768   3.169 0.001538 ** 
## ppt              4.5416     0.6527   6.958 3.98e-12 ***
## tmean            6.3042     1.4102   4.470 8.01e-06 ***
## tnt          -1558.5917   556.1444  -2.802 0.005094 ** 
## tri             45.4095    13.9735   3.250 0.001164 ** 
## tpi            -58.3009    16.2560  -3.586 0.000339 ***
## ndvi            -0.4211     0.1575  -2.673 0.007546 ** 
## northness       -5.7528     2.3299  -2.469 0.013582 *  
## eastness         5.1093     2.2136   2.308 0.021042 *  
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## 
## Residual standard error: 10290 on 4273 degrees of freedom
## Multiple R-squared:  0.3498, Adjusted R-squared:  0.348 
## F-statistic: 191.6 on 12 and 4273 DF,  p-value: < 2.2e-16
```

```
# Get predictions
fwd_preds <- predict(model_fwd, newdata = test_data)

mean((test_data$VOL.BF - fwd_preds)^2) # Test MSE
```

```
## [1] 106500006
```

MSE = 106500006

```
# Best Subset Selection
# nvmax should be the total number of predictors in your data
best_subsets <- regsubsets(VOL.BF ~ ., data = train_data, nvmax = 18)

# View which variables are in the 'best' model of each size
summary_best <- summary(best_subsets)
print(summary_best$outmat)
```

```
##           SECTIONM333B SECTIONM333C SECTIONM333D tcc16 elev eastness northness
## 1  ( 1 )  " "          " "          " "          "*"   " "  " "      " "      
## 2  ( 1 )  " "          " "          "*"          "*"   " "  " "      " "      
## 3  ( 1 )  " "          " "          " "          "*"   " "  " "      " "      
## 4  ( 1 )  " "          " "          " "          "*"   " "  " "      " "      
## 5  ( 1 )  " "          " "          " "          "*"   " "  " "      " "      
## 6  ( 1 )  " "          " "          "*"          "*"   " "  " "      " "      
## 7  ( 1 )  " "          " "          "*"          "*"   " "  " "      " "      
## 8  ( 1 )  " "          " "          "*"          "*"   " "  " "      " "      
## 9  ( 1 )  " "          " "          "*"          "*"   " "  " "      "*"      
## 10  ( 1 ) "*"          " "          "*"          "*"   " "  " "      "*"      
## 11  ( 1 ) "*"          " "          "*"          "*"   " "  "*"      "*"      
## 12  ( 1 ) "*"          " "          "*"          "*"   " "  "*"      "*"      
## 13  ( 1 ) "*"          " "          "*"          "*"   " "  "*"      "*"      
## 14  ( 1 ) "*"          " "          "*"          "*"   " "  "*"      "*"      
## 15  ( 1 ) "*"          " "          "*"          "*"   "*"  "*"      "*"      
## 16  ( 1 ) "*"          " "          "*"          "*"   "*"  "*"      "*"      
## 17  ( 1 ) "*"          "*"          "*"          "*"   "*"  "*"      "*"      
## 18  ( 1 ) "*"          "*"          "*"          "*"   "*"  "*"      "*"      
##           tri tpi ppt tmean tmax tmin tmin01 vpdmax tdmean def tnt evi ndvi
## 1  ( 1 )  " " " " " " " "   " "  " "  " "    " "    " "    " " " " " " " " 
## 2  ( 1 )  " " " " " " " "   " "  " "  " "    " "    " "    " " " " " " " " 
## 3  ( 1 )  " " " " "*" "*"   " "  " "  " "    " "    " "    " " " " " " " " 
## 4  ( 1 )  " " " " "*" "*"   " "  " "  " "    " "    " "    " " "*" " " " " 
## 5  ( 1 )  "*" " " "*" "*"   " "  " "  " "    " "    " "    " " "*" " " " " 
## 6  ( 1 )  " " "*" "*" " "   " "  "*"  " "    " "    " "    " " " " " " "*" 
## 7  ( 1 )  "*" "*" "*" " "   " "  " "  " "    " "    "*"    " " "*" " " " " 
## 8  ( 1 )  "*" "*" "*" " "   " "  " "  " "    " "    "*"    " " "*" " " "*" 
## 9  ( 1 )  "*" "*" "*" " "   " "  " "  " "    " "    "*"    " " "*" " " "*" 
## 10  ( 1 ) "*" "*" "*" "*"   " "  " "  " "    " "    " "    " " "*" " " "*" 
## 11  ( 1 ) "*" "*" "*" "*"   " "  " "  " "    " "    " "    " " "*" " " "*" 
## 12  ( 1 ) "*" "*" "*" "*"   " "  " "  " "    "*"    " "    " " "*" " " "*" 
## 13  ( 1 ) "*" "*" "*" "*"   " "  " "  "*"    "*"    " "    " " "*" " " "*" 
## 14  ( 1 ) "*" "*" "*" "*"   " "  " "  "*"    "*"    " "    "*" "*" " " "*" 
## 15  ( 1 ) "*" "*" "*" " "   "*"  "*"  "*"    "*"    " "    " " "*" " " "*" 
## 16  ( 1 ) "*" "*" "*" " "   "*"  "*"  "*"    "*"    " "    "*" "*" " " "*" 
## 17  ( 1 ) "*" "*" "*" " "   "*"  "*"  "*"    "*"    " "    "*" "*" " " "*" 
## 18  ( 1 ) "*" "*" "*" " "   "*"  "*"  "*"    "*"    " "    "*" "*" "*" "*"
```

```
# Best Subset Results

# Find the model size with the best metrics
which.max(summary_best$adjr2) # Highest Adjusted R-squared
```

```
## [1] 13
```

```
which.min(summary_best$cp)    # Lowest Mallows' Cp
```

```
## [1] 11
```

```
which.min(summary_best$bic)   # Lowest Bayesian Information Criterion
```

```
## [1] 7
```

```
best_coefs <- coef(best_subsets, id = 13)
names(best_coefs)
```

```
##  [1] "(Intercept)"  "SECTIONM333B" "SECTIONM333D" "tcc16"        "eastness"    
##  [6] "northness"    "tri"          "tpi"          "ppt"          "tmean"       
## [11] "tmin01"       "vpdmax"       "tnt"          "ndvi"
```

```
model1 <- lm(VOL.BF ~ SECTION + tcc16 + eastness + northness + tri + tpi + ppt + tmean + tmin01 + vpdmax + tnt +
ndvi, data = train_data)
summary(model1)
```

```
## 
## Call:
## lm(formula = VOL.BF ~ SECTION + tcc16 + eastness + northness + 
##     tri + tpi + ppt + tmean + tmin01 + vpdmax + tnt + ndvi, data = train_data)
## 
## Residuals:
##    Min     1Q Median     3Q    Max 
## -26007  -6248  -1343   3378  66367 
## 
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)    
## (Intercept)  -1.333e+04  4.078e+03  -3.268 0.001093 ** 
## SECTIONM333B  9.410e+02  4.805e+02   1.958 0.050275 .  
## SECTIONM333C -1.564e+02  6.429e+02  -0.243 0.807843    
## SECTIONM333D  1.811e+03  5.012e+02   3.613 0.000306 ***
## tcc16         3.148e+02  1.091e+01  28.857  < 2e-16 ***
## eastness      5.129e+00  2.214e+00   2.317 0.020552 *  
## northness    -5.637e+00  2.334e+00  -2.415 0.015774 *  
## tri           4.497e+01  1.407e+01   3.197 0.001399 ** 
## tpi          -5.881e+01  1.628e+01  -3.612 0.000307 ***
## ppt           4.675e+00  7.969e-01   5.866  4.8e-09 ***
## tmean         1.460e+01  4.921e+00   2.967 0.003027 ** 
## tmin01       -4.556e+00  3.212e+00  -1.418 0.156138    
## vpdmax       -4.224e+00  2.295e+00  -1.841 0.065739 .  
## tnt          -1.508e+03  5.571e+02  -2.706 0.006831 ** 
## ndvi         -4.700e-01  1.598e-01  -2.941 0.003289 ** 
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## 
## Residual standard error: 10290 on 4271 degrees of freedom
## Multiple R-squared:  0.3503, Adjusted R-squared:  0.3482 
## F-statistic: 164.5 on 14 and 4271 DF,  p-value: < 2.2e-16
```

```
# Get predictions
preds <- predict(model1, newdata = test_data)

mean((test_data$VOL.BF - preds)^2) # Test MSE
```

```
## [1] 106336437
```

MSE = 106336437

```
# Comparing Model Results Thus Far
models_list <- list(
  "Intercept Only" = model_simple,
  "Full" = model_full,
  "Backward Selection" = model_bwd,
  "Forward Selection" = model_fwd,
  "Best Subset Selection" = model1
)

modelsummary(
  models_list,
  stars = TRUE,
  gof_map = c("nobs", "r.squared")
)
```

	Intercept Only	Full	Backward Selection	Forward Selection	Best Subset Selection
(Intercept)	10364.024***	-18585.816**	-8598.921***	-9714.365***	-13325.428**
	(194.744)	(7064.645)	(2108.453)	(1503.435)	(4078.129)
SECTIONM333B		849.801+	986.801*	937.785*	940.967+
		(508.014)	(478.878)	(474.472)	(480.533)
SECTIONM333C		-414.531	-104.748	-233.447	-156.369
		(696.415)	(624.780)	(601.135)	(642.900)
SECTIONM333D		1551.643**	1560.764**	1497.809**	1811.157***
		(559.802)	(479.892)	(472.577)	(501.224)
tcc16		312.511***	314.916***	314.616***	314.820***
		(11.547)	(10.919)	(10.911)	(10.910)
elev		1.637			
		(1.689)			
eastness		4.959*	5.085*	5.109*	5.129*
		(2.221)	(2.214)	(2.214)	(2.214)
northness		-6.078*	-5.681*	-5.753*	-5.637*
		(2.395)	(2.332)	(2.330)	(2.334)
tri		45.568**	43.551**	45.409**	44.972**
		(14.369)	(14.190)	(13.974)	(14.067)
tpi		-63.870***	-58.836***	-58.301***	-58.813***
		(17.035)	(16.272)	(16.256)	(16.283)
ppt		4.410***	4.365***	4.542***	4.675***
		(0.918)	(0.693)	(0.653)	(0.797)
tmean		-119.389		6.304***	14.600**
		(391.295)		(1.410)	(4.922)
tmax		73.670	2.304+		
		(195.589)	(1.326)		
tmin		67.199	4.222**		
		(195.744)	(1.582)		
tmin01		-4.856			-4.556
		(3.412)			(3.212)
vpdmax		-7.415			-4.224+
		(6.300)			(2.295)
tdmean		-1.089			
		(4.567)			
def		-2.514			
		(3.502)			
tnt		-1584.388**	-1555.038**	-1558.592**	-1507.678**
		(560.919)	(556.193)	(556.144)	(557.100)
evi		0.030			
		(0.075)			
ndvi		-0.547*	-0.424**	-0.421**	-0.470**
		(0.236)	(0.158)	(0.158)	(0.160)
Num.Obs.	4286	4286	4286	4286	4286
R2	0.000	0.351	0.350	0.350	0.350

+ p < 0.1, * p < 0.05, ** p < 0.01, *** p < 0.001

It does not appear that any selection method makes a significant difference in R-squared from the full model.

===

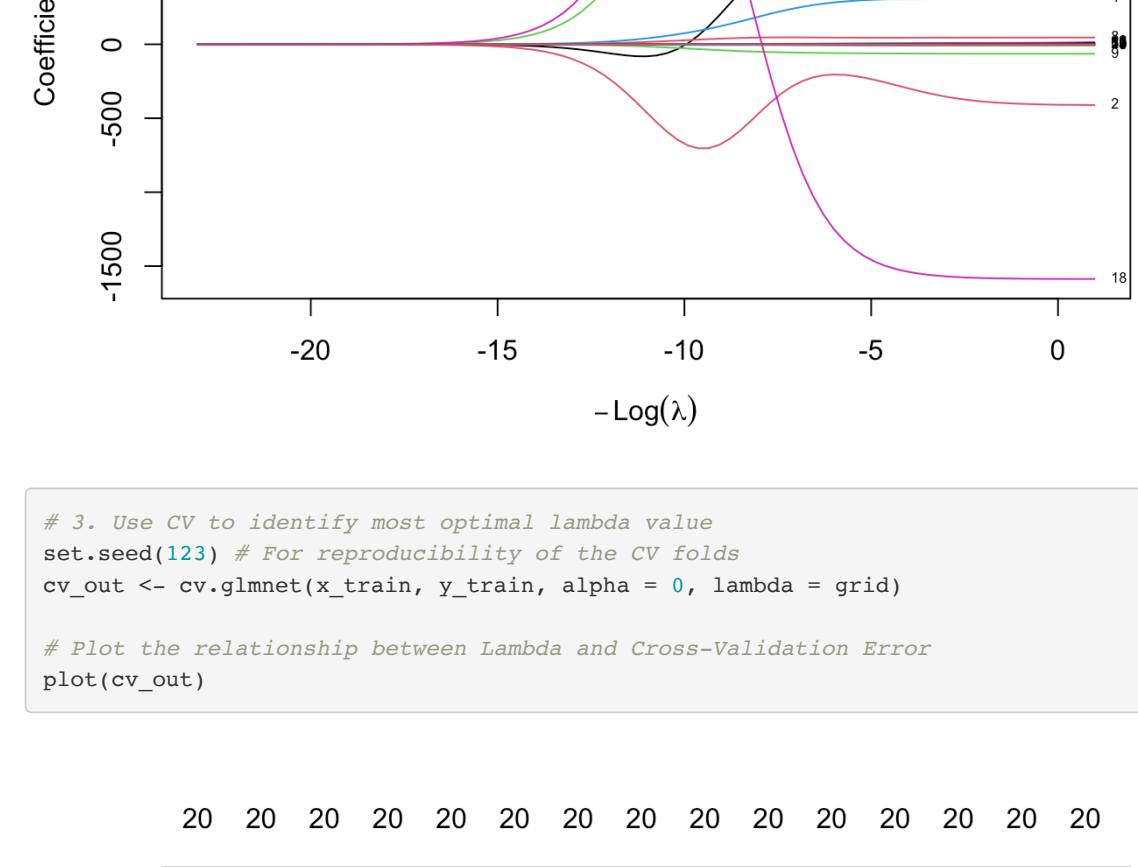
Ridge/Lasso Regression

```
# RIDGE REGRESSION

# 1. Data Preparation

# Create numeric matrices for predictors (dropping the Intercept column with [, -1])
# We use the formula of our desired model (e.g., VOL.BF ~ .)
x_train <- model.matrix(VOL.BF ~ ., data = train_data)[, -1]
y_train <- train_data$VOL.BF

x_test <- model.matrix(VOL.BF ~ ., data = test_data)[, -1]
y_test <- test_data$VOL.BF

# Define a grid of lambda values to test (from 10^10 to 10^-2)
grid <- 10^seq(10, -2, length = 100)

# 2. Fit Initial Ridge Model

# Fit ridge regression models for all lambdas in the grid
ridge_mod <- glmnet(x_train, y_train, alpha = 0, lambda = grid, thresh = 1e-12)
```

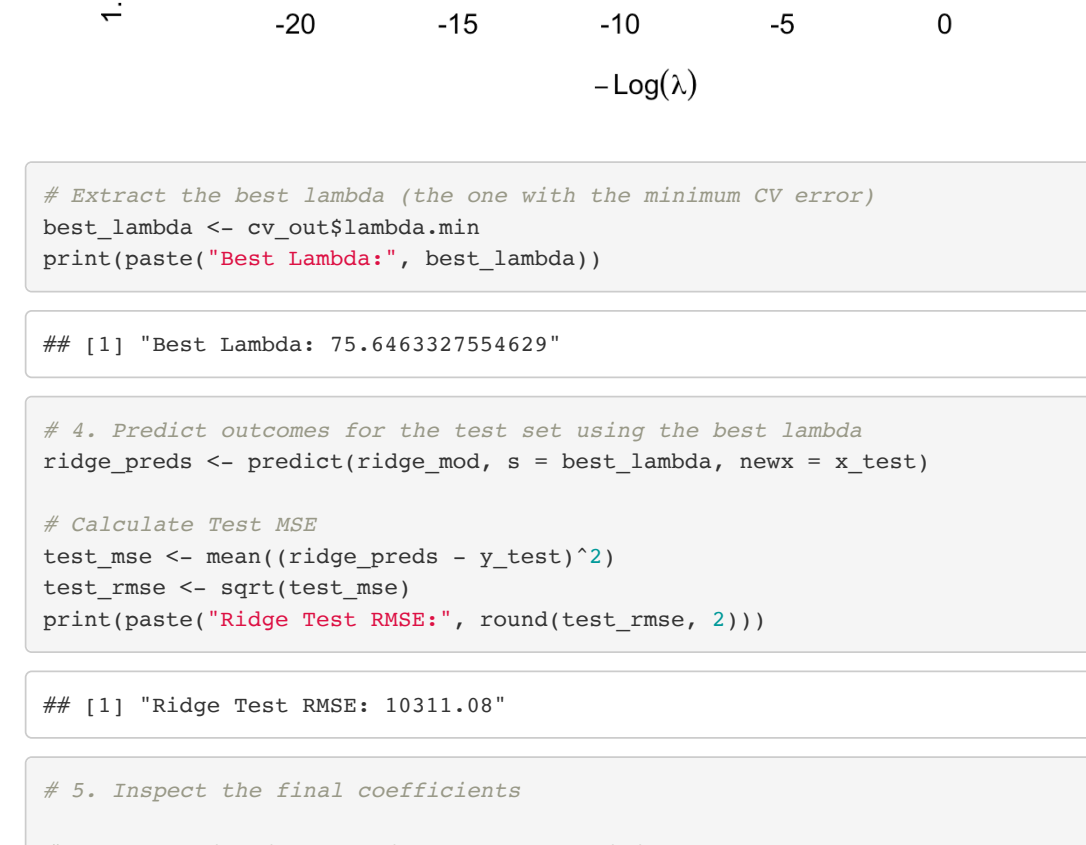
```
## Warning in .resolve_control(control = control, nvars = nvars, deprecated =
## list(thresh = if (!missing(thresh)) thresh, : Passing 'thresh' to glmnet() is
## deprecated. Use control = list(thresh = ...) instead.
```

```
## Warning: from glmnet C++ code (error code -88); Convergence for 88th lambda
## value not reached after maxit=100000 iterations; solutions for larger lambdas
## returned
```

```
# Optional: Visualize how coefficients shrink as lambda increases
plot(ridge_mod, xvar = "lambda", label = TRUE)
```



```
# 3. Use CV to identify most optimal lambda value
set.seed(123) # For reproducibility of the CV folds
cv_out <- cv.glmnet(x_train, y_train, alpha = 0, lambda = grid)

# Plot the relationship between Lambda and Cross-Validation Error
plot(cv_out)
```



```
# Extract the best lambda (the one with the minimum CV error)
best_lambda <- cv_out$lambda.min
print(paste("Best Lambda:", best_lambda))
```

```
## [1] "Best Lambda: 75.6463327554629"
```

```
# 4. Predict outcomes for the test set using the best lambda
ridge_preds <- predict(ridge_mod, s = best_lambda, newx = x_test)

# Calculate Test MSE
test_mse <- mean((ridge_preds - y_test)^2)
test_rmse <- sqrt(test_mse)
print(paste("Ridge Test RMSE:", round(test_rmse, 2)))
```

```
## [1] "Ridge Test RMSE: 10311.08"
```

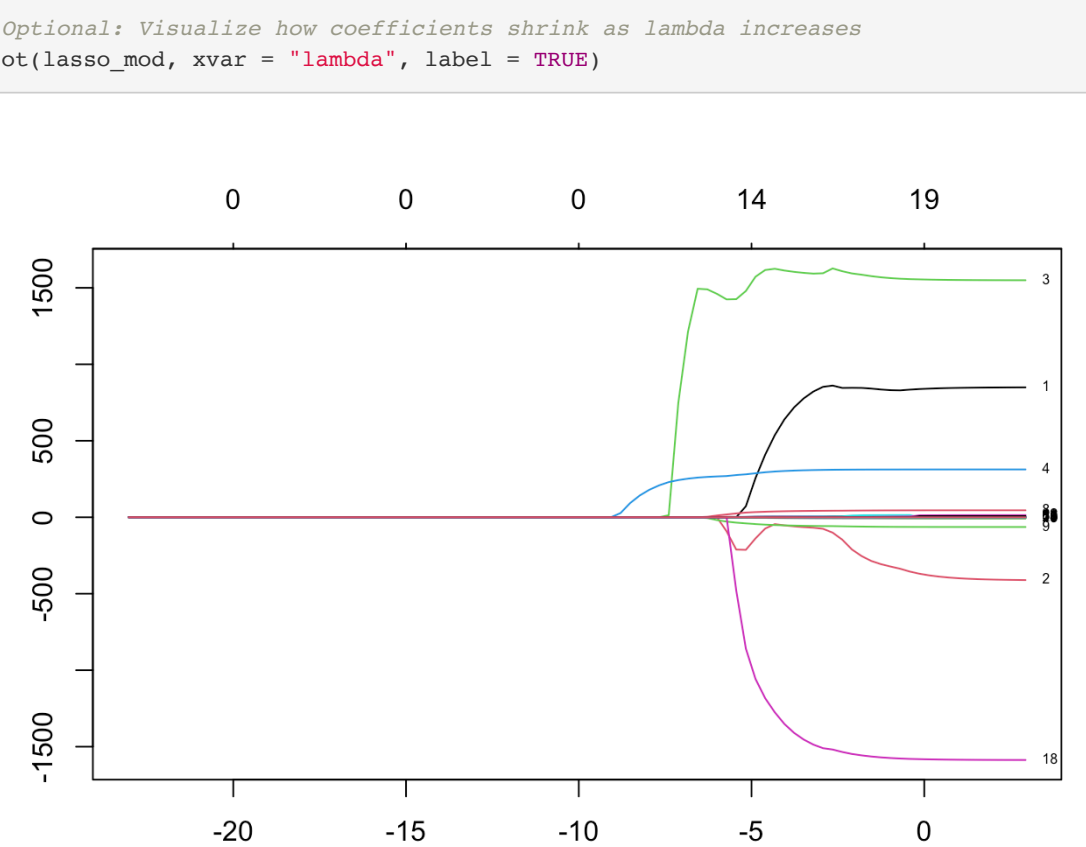
```
# 5. Inspect the final coefficients

# Note that in Ridge, unlike Lasso, coefficients are shrunk toward zero
# but rarely become exactly zero.
final_coefs <- predict(ridge_mod, type = "coefficients", s = best_lambda)
print(final_coefs)
```

```
## 21 x 1 sparse Matrix of class "dgCMatrix"
##                       s=75.64633
## (Intercept)   -1.393058e+04
## SECTIONM333B   8.190763e+02
## SECTIONM333C  -2.770273e+02
## SECTIONM333D   1.590230e+03
## tcc16          3.062370e+02
## eastness        4.862327e+00
## northness      -6.012849e+00
## tri             4.475188e+01
## tpi            -6.221241e+01
## ppt             4.106372e+00
## tmean           5.156807e+00
## tmax            4.775122e+00
## tmin            4.082721e+00
## tmin01         -3.527113e+00
## vpdmax         -2.767385e+00
## tdmean          8.217066e-01
## def            -3.073335e+00
## tnt            -1.520531e+03
## evi             3.300209e-02
## ndvi           -4.856789e-01
```

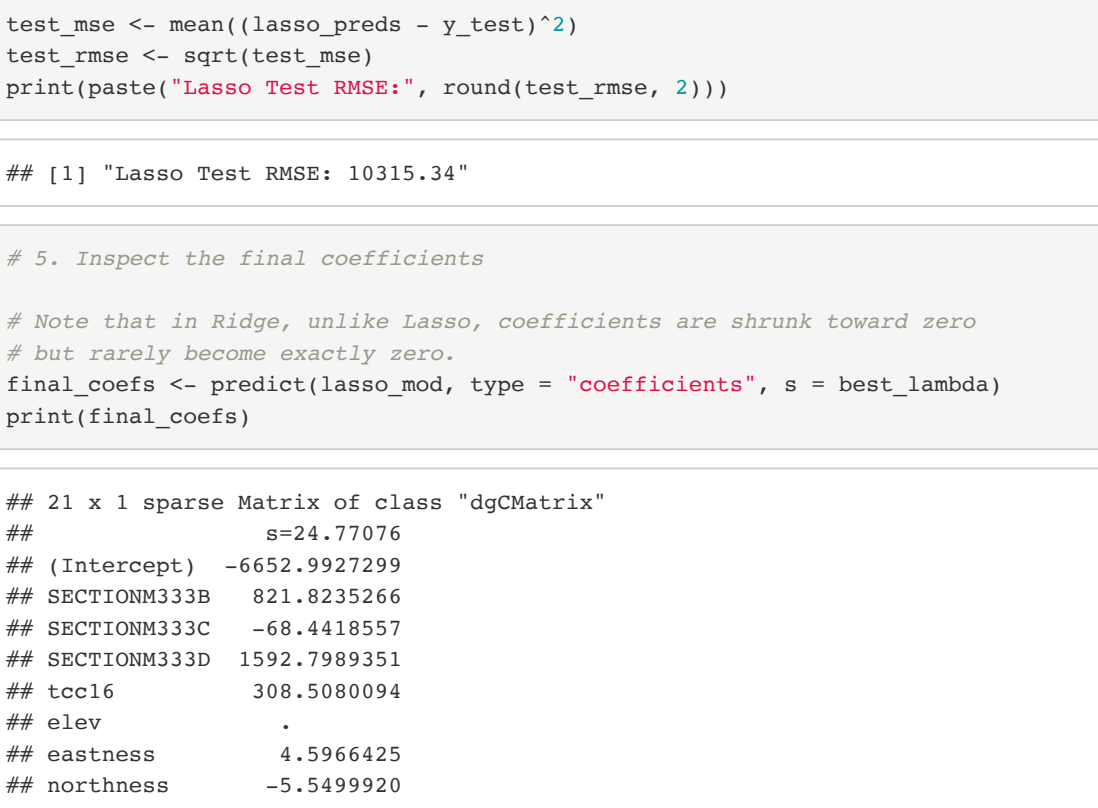
```
# LASSO REGRESSION

# 1. Data Preparation

# Create numeric matrices for predictors (dropping the Intercept column with [, -1])
# We use the formula of our desired model (e.g., VOL.BF ~ .)
x_train <- model.matrix(VOL.BF ~ ., data = train_data)[, -1]
y_train <- train_data$VOL.BF

x_test <- model.matrix(VOL.BF ~ ., data = test_data)[, -1]
y_test <- test_data$VOL.BF

# Define a grid of lambda values to test (from 10^10 to 10^-2)
grid <- 10^seq(10, -2, length = 100)

# 2. Fit Initial Lasso Model

# Fit lasso regression models for all lambdas in the grid
lasso_mod <- glmnet(x_train, y_train, alpha = 1, lambda = grid, thresh = 1e-12)
```

```
## Warning: from glmnet C++ code (error code -95); Convergence for 95th lambda
## value not reached after maxit=100000 iterations; solutions for larger lambdas
## returned
```

```
# Optional: Visualize how coefficients shrink as lambda increases
plot(lasso_mod, xvar = "lambda", label = TRUE)
```



```
# 3. Use CV to identify most optimal lambda value
set.seed(123) # For reproducibility of the CV folds
cv_out <- cv.glmnet(x_train, y_train, alpha = 1, lambda = grid)

# Plot the relationship between Lambda and Cross-Validation Error
plot(cv_out)
```



```
# Extract the best lambda (the one with the minimum CV error)
best_lambda <- cv_out$lambda.min
print(paste("Best Lambda:", best_lambda))
```

```
## [1] "Best Lambda: 24.7707635599171"
```

```
# 4. Predict outcomes for the test set using the best lambda
lasso_preds <- predict(lasso_mod, s = best_lambda, newx = x_test)

# Calculate Test MSE
test_mse <- mean((lasso_preds - y_test)^2)
test_rmse <- sqrt(test_mse)
print(paste("Lasso Test RMSE:", round(test_rmse, 2)))
```

```
## [1] "Lasso Test RMSE: 10315.34"
```

```
# 5. Inspect the final coefficients

# Note that in Ridge, unlike Lasso, coefficients are shrunk toward zero
# but rarely become exactly zero.
final_coefs <- predict(lasso_mod, type = "coefficients", s = best_lambda)
print(final_coefs)
```

```
## 21 x 1 sparse Matrix of class "dgCMatrix"
##                    s=24.77076
## (Intercept)  -6652.9927299
## SECTIONM333B   821.8235266
## SECTIONM333C   -68.4418557
## SECTIONM333D  1592.7989351
## tcc16          308.5080094
## elev             .
## eastness         4.5966425
## northness       -5.5499920
## tri             41.7021477
## tpi            -56.9874231
## ppt              3.6989326
## tmean            3.6287294
## tmax             .
## tmin             2.2457071
## tmin01           .
## vpdmax           .
## tdmean           0.6774602
## def             -2.2713066
## tnt          -1486.5976815
## evi              .
## ndvi            -0.3747790
```

==

Random Forest

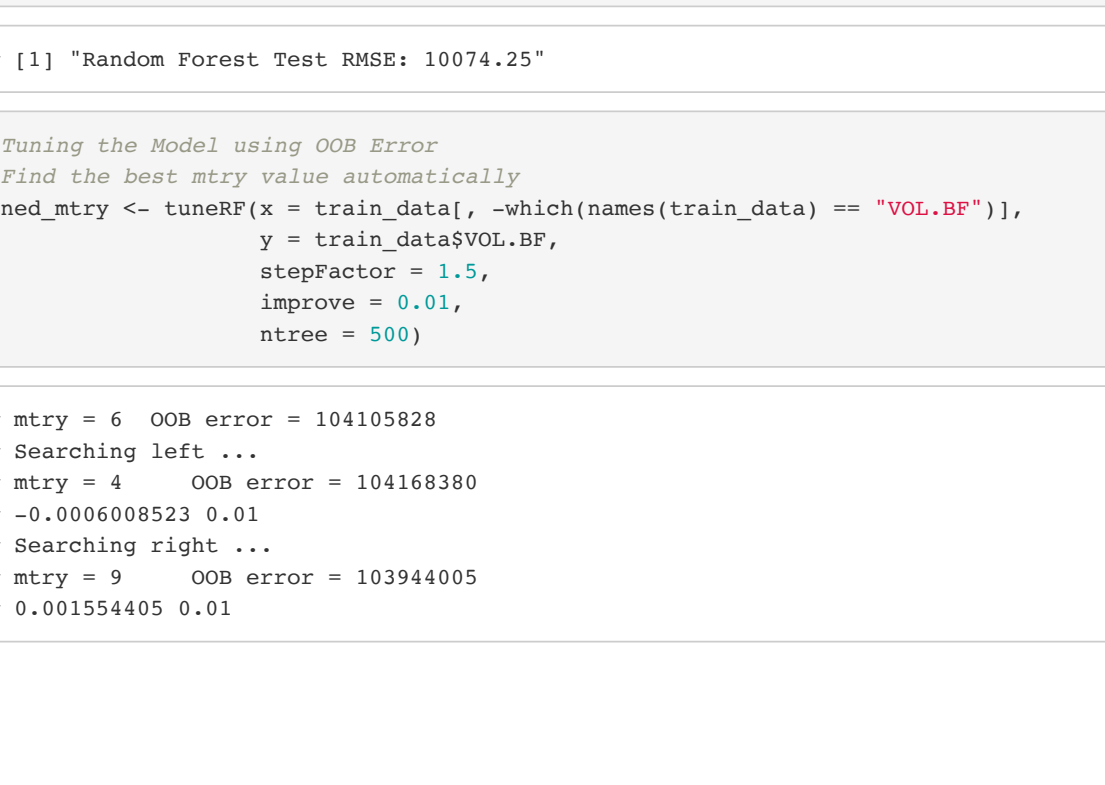
```
# 1. Fit the Random Forest Model

# Set seed for reproducibility because RF uses random sampling
set.seed(123)


# Parameters:
# mtry: Subset of randomly sampled variables to be candidates at each split.
#       For regression, the default is p/3 (p = 18, so mtry = 6)
# ntree: Number of trees to grow. 500 is usually enough.
# importance: Set to TRUE so we can see which predictors matter most.

# Fit the model
rf_model <- randomForest(VOL.BF ~ .,
                          data = train_data,
                          ntree = 500,
                          mtry = 6,
                          importance = TRUE)

# View the basic model output
print(rf_model)
```

```
##
## Call:
##  randomForest(formula = VOL.BF ~ ., data = train_data, ntree = 500,      mtry = 6, importance = TRUE)
##                Type of random forest: regression
##                      Number of trees: 500
## No. of variables tried at each split: 6
##
##           Mean of squared residuals: 103694892
##                     % Var explained: 36.19
```

```
# 2. View variable importance scores
importance(rf_model)
```

```
##             %IncMSE IncNodePurity
## SECTION    9.104651    8210469519
## tcc16     63.110668  158952122195
## elev      19.207670   23994231388
## eastness   4.966982   25154692639
## northness 13.596545   27858439535
## tri       13.410829   32757018399
## tpi        5.904360   32532078252
## ppt       18.782021   41305501468
## tmean     19.863669   21990089141
## tmax      19.781496   21665267730
## tmin      18.840398   23846681726
## tmin01    18.413323   27545028168
## vpdmax    18.660723   21423106638
## tdmean    16.008843   25533389343
## def       15.084488   28732274752
## tnt        1.080486   12346175875
## evi        9.324614   66110783552
## ndvi       9.298723   67982615614
```

```
# Plot importance
varImpPlot(rf_model, main = "Variable Importance for Forest Volume")
```

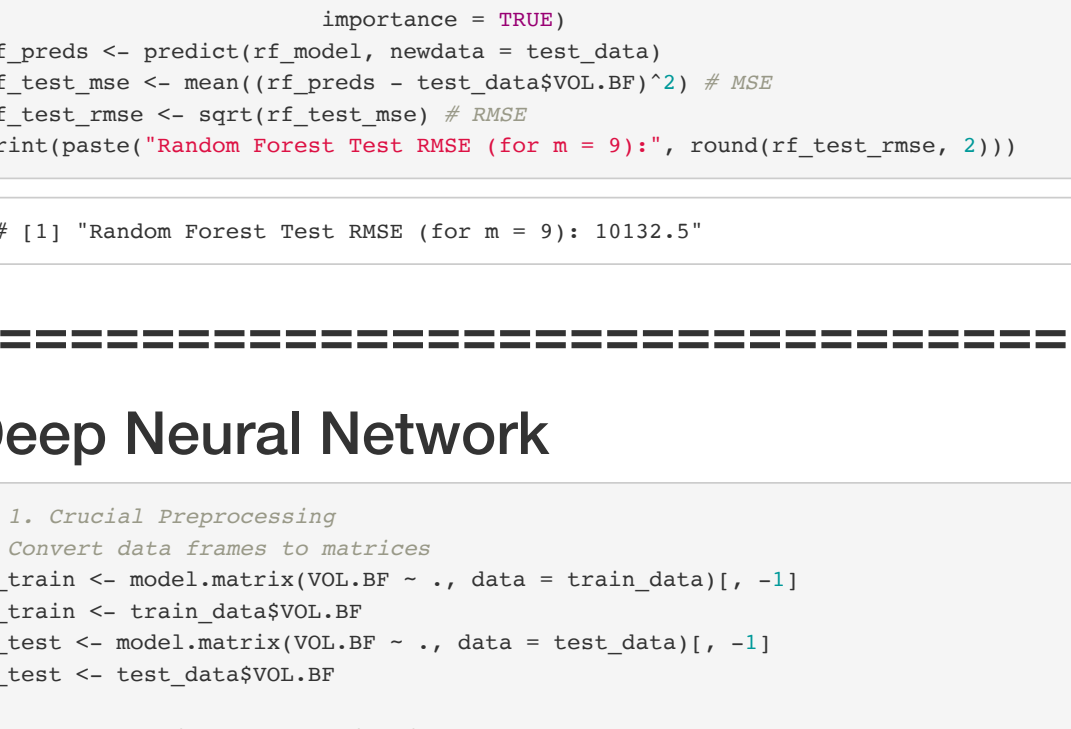


```
# 3. Make Predictions and Calculate Test Error
rf_preds <- predict(rf_model, newdata = test_data)

# Calculate Test Mean Squared Error (MSE)
rf_test_mse <- mean((rf_preds - test_data$VOL.BF)^2)

# Calculate Root Mean Squared Error (RMSE) for interpretability
# (This is the error in the actual units of VOL.BF)
rf_test_rmse <- sqrt(rf_test_mse)

print(paste("Random Forest Test RMSE:", round(rf_test_rmse, 2)))
```

```
## [1] "Random Forest Test RMSE: 10074.25"
```

```
# Tuning the Model using OOB Error
# Find the best mtry value automatically
tuned_mtry <- tuneRF(x = train_data[, -which(names(train_data) == "VOL.BF")],
                     y = train_data$VOL.BF,
                     stepFactor = 1.5,
                     improve = 0.01,
                     ntree = 500)
```

```
## mtry = 6  OOB error = 104105828
## Searching left ...
## mtry = 4     OOB error = 104168380
## -0.0006008523 0.01
## Searching right ...
## mtry = 9     OOB error = 103944005
## 0.001554405 0.01
```



```
print("Optimal m = 9")
```

```
## [1] "Optimal m = 9"
```

```
rf_model <- randomForest(VOL.BF ~ .,
                        data = train_data,
                        ntree = 500,
                        mtry = 9,
                        importance = TRUE)
rf_preds <- predict(rf_model, newdata = test_data)
rf_test_mse <- mean((rf_preds - test_data$VOL.BF)^2) # MSE
rf_test_rmse <- sqrt(rf_test_mse) # RMSE
print(paste("Random Forest Test RMSE (for m = 9):", round(rf_test_rmse, 2)))
```

```
## [1] "Random Forest Test RMSE (for m = 9): 10132.5"
```

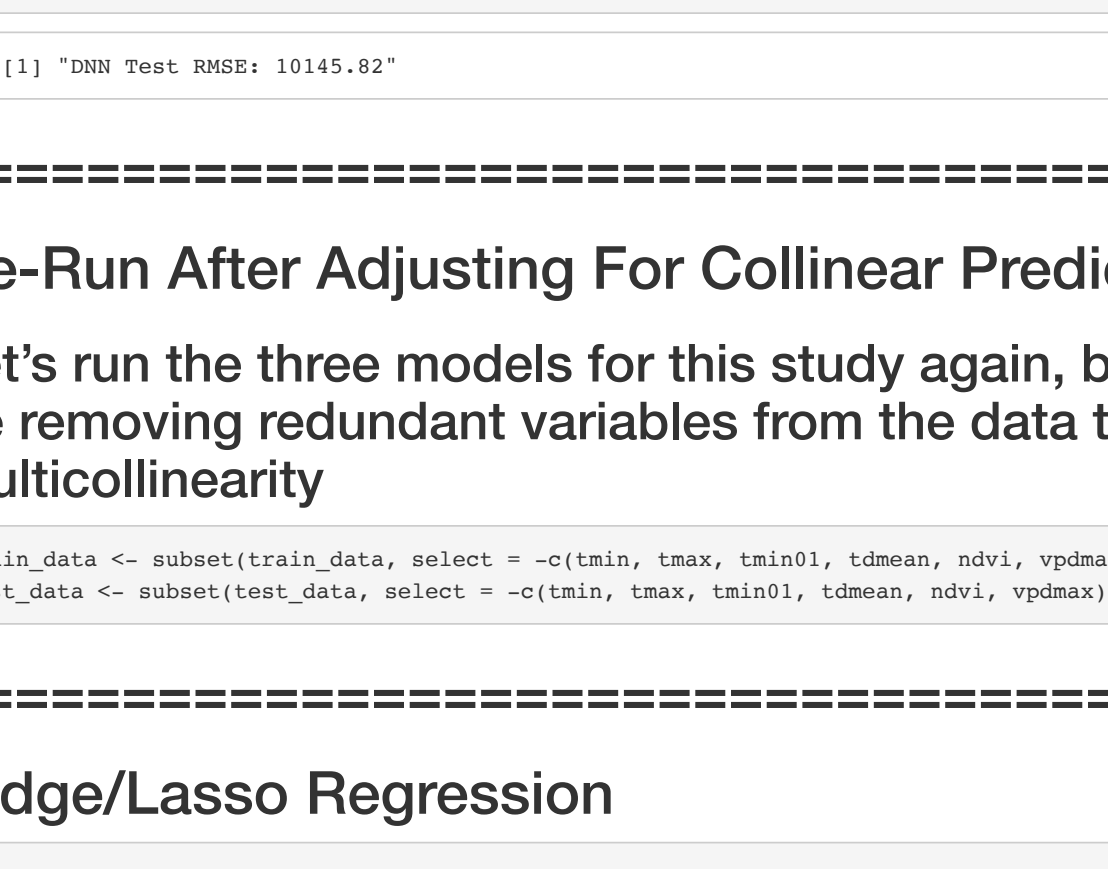
==

Deep Neural Network

```
# 1. Crucial Preprocessing
# Convert data frames to matrices
x_train <- model.matrix(VOL.BF ~ ., data = train_data)[, -1]
y_train <- train_data$VOL.BF
x_test <- model.matrix(VOL.BF ~ ., data = test_data)[, -1]
y_test <- test_data$VOL.BF

# Feature Scaling (Standardization)
# We calculate mean/sd from training and apply to both sets
mean <- colMeans(x_train) # calculating means of each column from x_train matrix
std <- apply(x_train, 2, sd) # calculating sd for each column (2 tells apply() to work with columns)

# Scaling the training variables
x_train <- scale(x_train, center = mean, scale = std)
x_test <- scale(x_test, center = mean, scale = std)

# 2. Define Architecture
model <- keras_model_sequential() %>%
  # Layer 1: Input layer (must match your 18 predictors)
  layer_dense(units = 64, activation = "relu", input_shape = ncol(x_train)) %>%
  # Layer 2: Hidden layer
  layer_dense(units = 32, activation = "relu") %>%
  # Layer 3: Hidden layer
  layer_dense(units = 16, activation = "relu") %>%
  # Output layer: 1 unit for Regression (no activation function for raw numeric output)
  layer_dense(units = 1)

# Compile the model
model %>% compile(
  optimizer = "rmsprop",
  loss = "mse",       # We optimize for Mean Squared Error
  metrics = c("mae") # We track Mean Absolute Error (easier to read)
)

# 3. Training the Model
history <- model %>% fit(
  x_train, y_train,
  epochs = 50,
  batch_size = 32,
  validation_split = 0.2, # Automatically uses 20% of training data for "OOB-like" checks
  verbose = 1
)
```

```
## Epoch 1/50
##   1/108 [..............................] - ETA: 10:09 - loss: 159042944.0000 - mae: 8031.8779
 87/108 [=======================
>......] - ETA: 0s - loss: 223111520.0000 - mae: 9352.7812   
108/108 [==============================] - 6s 2ms/step - loss: 230113808.00
00 - mae: 9513.2041 - val_loss: 425436864.0000 - val_mae: 13671.5830
## Epoch 2/50
##   1/108 [..............................] - ETA: 0s - loss: 314333760.0000 - mae: 11185.0625
108/108 [==============================]
- 0s 742us/step - loss: 226007488.0000 - mae: 9397.0801 - val_loss: 415987040.0000 - val_mae: 13438.3438
## Epoch 3/50
##   1/108 [..............................] - ETA: 0s - loss: 206571936.0000 - mae: 9310.4512
108/108 [==============================] -
0s 667us/step - loss: 212155568.0000 - mae: 9038.9951 - val_loss: 388831040.0000 - val_mae: 12804.9902
## Epoch 4/50
##   1/108 [..............................] - ETA: 0s - loss: 139601168.0000 - mae: 6538.2178
108/108 [==============================] -
0s 642us/step - loss: 184665456.0000 - mae: 8410.4873 - val_loss: 339663296.0000 - val_mae: 11738.1016
## Epoch 5/50
##   1/108 [..............................] - ETA: 0s - loss: 566301696.0000 - mae: 13855.8662
108/108 [==============================]
- 0s 634us/step - loss: 149632976.0000 - mae: 7695.2515 - val_loss: 270848768.0000 - val_mae: 10413.7432
## Epoch 6/50
##   1/108 [..............................] - ETA: 0s - loss: 83139984.0000 - mae: 5866.8345
108/108 [==============================] - 0s
637us/step - loss: 117806600.0000 - mae: 7161.6777 - val_loss: 216382272.0000 - val_mae: 9510.6670
## Epoch 7/50
##   1/108 [..............................] - ETA: 0s - loss: 145911792.0000 - mae: 8253.3906
108/108 [==============================] -
0s 628us/step - loss: 102741216.0000 - mae: 6952.2134 - val_loss: 193890544.0000 - val_mae: 9267.9229
## Epoch 8/50
##   1/108 [..............................] - ETA: 0s - loss: 79120408.0000 - mae: 5790.0112
108/108 [==============================] - 0s
650us/step - loss: 97860176.0000 - mae: 6805.9473 - val_loss: 187394864.0000 - val_mae: 9200.8389
## Epoch 9/50
##   1/108 [..............................] - ETA: 0s - loss: 79880272.0000 - mae: 6034.5059
108/108 [==============================] - 0s
676us/step - loss: 95916952.0000 - mae: 6676.6953 - val_loss: 183817232.0000 - val_mae: 9155.8994
## Epoch 10/50
##   1/108 [..............................] - ETA: 0s - loss: 169099264.0000 - mae: 8000.3828
108/108 [==============================] -
0s 667us/step - loss: 94716360.0000 - mae: 6611.0259 - val_loss: 180933472.0000 - val_mae: 9146.2070
## Epoch 11/50
##   1/108 [..............................] - ETA: 0s - loss: 92016112.0000 - mae: 6044.2783
108/108 [==============================] - 0s
664us/step - loss: 93746152.0000 - mae: 6578.6865 - val_loss: 179467376.0000 - val_mae: 9132.9902
## Epoch 12/50
##   1/108 [..............................] - ETA: 0s - loss: 160835536.0000 - mae: 8723.3574
108/108 [==============================] -
0s 650us/step - loss: 93028168.0000 - mae: 6535.4556 - val_loss: 178542208.0000 - val_mae: 9095.9131
## Epoch 13/50
##   1/108 [..............................] - ETA: 0s - loss: 83043920.0000 - mae: 5841.8926
 98/108 [==========================>...] - ET
A: 0s - loss: 92731664.0000 - mae: 6503.1372
108/108 [==============================] - 0s 726us/step - loss: 92429680.0000 - mae: 6481.87
30 - val_loss: 177532960.0000 - val_mae: 9070.6553
## Epoch 14/50
##   1/108 [..............................] - ETA: 0s - loss: 141704960.0000 - mae: 8254.3105
108/108 [==============================] -
0s 671us/step - loss: 91927280.0000 - mae: 6465.0332 - val_loss: 175601408.0000 - val_mae: 9112.1641
## Epoch 15/50
##   1/108 [..............................] - ETA: 0s - loss: 136723456.0000 - mae: 7990.0200
108/108 [==============================] -
0s 661us/step - loss: 91429672.0000 - mae: 6453.1685 - val_loss: 175779904.0000 - val_mae: 9048.4141
## Epoch 16/50
##   1/108 [..............................] - ETA: 0s - loss: 104845800.0000 - mae: 6812.2290
108/108 [==============================] -
0s 656us/step - loss: 91065248.0000 - mae: 6418.1104 - val_loss: 174772304.0000 - val_mae: 9063.7002
## Epoch 17/50
##   1/108 [..............................] - ETA: 0s - loss: 61376308.0000 - mae: 5668.2656
108/108 [==============================] - 0s
662us/step - loss: 90649992.0000 - mae: 6396.5371 - val_loss: 173382768.0000 - val_mae: 9098.3486
## Epoch 18/50
##   1/108 [..............................] - ETA: 0s - loss: 70714808.0000 - mae: 6199.4727
108/108 [==============================] - 0s
661us/step - loss: 90320032.0000 - mae: 6412.9380 - val_loss: 173555024.0000 - val_mae: 9047.0068
## Epoch 19/50
##   1/108 [..............................] - ETA: 0s - loss: 80802864.0000 - mae: 6009.4336
108/108 [==============================] - 0s
662us/step - loss: 89931192.0000 - mae: 6368.7935 - val_loss: 172148768.0000 - val_mae: 9177.6621
## Epoch 20/50
##   1/108 [..............................] - ETA: 0s - loss: 92564408.0000 - mae: 6890.2969
108/108 [==============================] - 0s
662us/step - loss: 89865696.0000 - mae: 6413.8838 - val_loss: 172025088.0000 - val_mae: 9065.2334
## Epoch 21/50
##   1/108 [..............................] - ETA: 0s - loss: 85147368.0000 - mae: 6152.6865
108/108 [==============================] - 0s
663us/step - loss: 89561968.0000 - mae: 6385.5996 - val_loss: 172067216.0000 - val_mae: 9034.7900
## Epoch 22/50
##   1/108 [..............................] - ETA: 0s - loss: 49766612.0000 - mae: 4918.1074
108/108 [==============================] - 0s
670us/step - loss: 89281856.0000 - mae: 6379.1436 - val_loss: 171616000.0000 - val_mae: 9034.4121
## Epoch 23/50
##   1/108 [..............................] - ETA: 0s - loss: 106241872.0000 - mae: 6605.0000
108/108 [==============================] -
0s 672us/step - loss: 89072928.0000 - mae: 6370.0947 - val_loss: 172131792.0000 - val_mae: 8976.7402
## Epoch 24/50
##   1/108 [..............................] - ETA: 0s - loss: 50075104.0000 - mae: 5240.3931
108/108 [==============================] - 0s
679us/step - loss: 88851568.0000 - mae: 6350.1265 - val_loss: 171844704.0000 - val_mae: 8950.2412
## Epoch 25/50
##   1/108 [..............................] - ETA: 0s - loss: 46653708.0000 - mae: 5183.4644
108/108 [==============================] - 0s
651us/step - loss: 88700392.0000 - mae: 6318.7627 - val_loss: 170078464.0000 - val_mae: 9038.6533
## Epoch 26/50
##   1/108 [..............................] - ETA: 0s - loss: 82581280.0000 - mae: 6417.1660
108/108 [==============================] - 0s
656us/step - loss: 88509648.0000 - mae: 6358.7300 - val_loss: 169872704.0000 - val_mae: 9098.3018
## Epoch 27/50
##   1/108 [..............................] - ETA: 0s - loss: 53426476.0000 - mae: 5645.2949
108/108 [==============================] - 0s
673us/step - loss: 88327472.0000 - mae: 6355.3438 - val_loss: 169536336.0000 - val_mae: 9041.6250
## Epoch 28/50
##   1/108 [..............................] - ETA: 0s - loss: 116354128.0000 - mae: 6751.2041
108/108 [==============================] -
0s 656us/step - loss: 88190424.0000 - mae: 6329.9429 - val_loss: 169452048.0000 - val_mae: 9040.2061
## Epoch 29/50
##   1/108 [..............................] - ETA: 0s - loss: 89478768.0000 - mae: 6716.6558
108/108 [==============================] - 0s
660us/step - loss: 87981408.0000 - mae: 6330.3447 - val_loss: 169232912.0000 - val_mae: 9031.9531
## Epoch 30/50
##   1/108 [..............................] - ETA: 0s - loss: 159586976.0000 - mae: 7189.7881
108/108 [==============================] -
0s 666us/step - loss: 87854256.0000 - mae: 6337.5269 - val_loss: 169067264.0000 - val_mae: 9053.2812
## Epoch 31/50
##   1/108 [..............................] - ETA: 0s - loss: 106695040.0000 - mae: 7496.3247
104/108 [===========================>..] -
ETA: 0s - loss: 88094008.0000 - mae: 6366.9761 
108/108 [==============================] - 0s 699us/step - loss: 87707032.0000 - mae: 633
8.7515 - val_loss: 168793824.0000 - val_mae: 9066.1162
## Epoch 32/50
##   1/108 [..............................] - ETA: 0s - loss: 114912968.0000 - mae: 7226.8057
108/108 [==============================] -
0s 653us/step - loss: 87620256.0000 - mae: 6343.0063 - val_loss: 168865232.0000 - val_mae: 9001.0518
## Epoch 33/50
##   1/108 [..............................] - ETA: 0s - loss: 57504128.0000 - mae: 5677.6162
108/108 [==============================] - 0s
657us/step - loss: 87440128.0000 - mae: 6324.9150 - val_loss: 168519392.0000 - val_mae: 8997.7734
## Epoch 34/50
##   1/108 [..............................] - ETA: 0s - loss: 43389160.0000 - mae: 4632.7710
108/108 [==============================] - 0s
652us/step - loss: 87351600.0000 - mae: 6312.9648 - val_loss: 168188112.0000 - val_mae: 9031.2402
## Epoch 35/50
##   1/108 [..............................] - ETA: 0s - loss: 155361824.0000 - mae: 7415.3101
108/108 [==============================] -
0s 646us/step - loss: 87231992.0000 - mae: 6322.5298 - val_loss: 168121664.0000 - val_mae: 9034.6553
## Epoch 36/50
##   1/108 [..............................] - ETA: 0s - loss: 53284992.0000 - mae: 4692.6113
108/108 [==============================] - ET
A: 0s - loss: 87107704.0000 - mae: 6334.8501
108/108 [==============================] - 0s 696us/step - loss: 87107704.0000 - mae: 6334.85
01 - val_loss: 168289328.0000 - val_mae: 8998.0850
## Epoch 37/50
##   1/108 [..............................] - ETA: 0s - loss: 105619040.0000 - mae: 6467.5601
106/108 [============================>.] -
ETA: 0s - loss: 87542000.0000 - mae: 6337.4043 
108/108 [==============================] - 0s 689us/step - loss: 86985472.0000 - mae: 631
7.8096 - val_loss: 167951088.0000 - val_mae: 9047.6006
## Epoch 38/50
##   1/108 [..............................] - ETA: 0s - loss: 90839864.0000 - mae: 6679.8721
108/108 [==============================] - 0s
680us/step - loss: 86855176.0000 - mae: 6324.7583 - val_loss: 167674464.0000 - val_mae: 9079.1318
## Epoch 39/50
##   1/108 [..............................] - ETA: 0s - loss: 50080688.0000 - mae: 4364.3467
108/108 [==============================] - ET
A: 0s - loss: 86824224.0000 - mae: 6318.1597
108/108 [==============================] - 0s 700us/step - loss: 86824224.0000 - mae: 6318.15
97 - val_loss: 167524928.0000 - val_mae: 9073.7734
## Epoch 40/50
##   1/108 [..............................] - ETA: 0s - loss: 118337344.0000 - mae: 7130.8384
105/108 [============================>.] -
ETA: 0s - loss: 86499432.0000 - mae: 6328.8076 
108/108 [==============================] - 0s 710us/step - loss: 86751760.0000 - mae: 632
8.2627 - val_loss: 167350144.0000 - val_mae: 9056.2354
## Epoch 41/50
##   1/108 [..............................] - ETA: 0s - loss: 54886452.0000 - mae: 5457.4097
107/108 [============================>.] - ET
A: 0s - loss: 86698824.0000 - mae: 6327.7808
108/108 [==============================] - 0s 688us/step - loss: 86611760.0000 - mae: 6324.10
89 - val_loss: 167379360.0000 - val_mae: 9043.7090
## Epoch 42/50
##   1/108 [..............................] - ETA: 0s - loss: 66284424.0000 - mae: 5891.5200
108/108 [==============================] - 0s
666us/step - loss: 86490384.0000 - mae: 6323.5542 - val_loss: 167445104.0000 - val_mae: 8985.0312
## Epoch 43/50
##   1/108 [..............................] - ETA: 0s - loss: 33411350.0000 - mae: 4328.6626
108/108 [==============================] - 0s
670us/step - loss: 86479992.0000 - mae: 6304.6353 - val_loss: 167286304.0000 - val_mae: 9015.2520
## Epoch 44/50
##   1/108 [..............................] - ETA: 0s - loss: 46355344.0000 - mae: 4925.1021
108/108 [==============================] - 0s
678us/step - loss: 86339328.0000 - mae: 6288.6260 - val_loss: 167094160.0000 - val_mae: 9020.7363
## Epoch 45/50
##   1/108 [..............................] - ETA: 0s - loss: 93776272.0000 - mae: 6389.7451
108/108 [==============================] - 0s
661us/step - loss: 86277544.0000 - mae: 6320.6875 - val_loss: 167275696.0000 - val_mae: 8996.4990
## Epoch 46/50
##   1/108 [..............................] - ETA: 0s - loss: 67323736.0000 - mae: 5698.3809
108/108 [==============================] - 0s
671us/step - loss: 86225640.0000 - mae: 6301.0767 - val_loss: 167215872.0000 - val_mae: 9012.6338
## Epoch 47/50
##   1/108 [..............................] - ETA: 0s - loss: 82036784.0000 - mae: 5886.4189
108/108 [==============================] - 0s
658us/step - loss: 86152272.0000 - mae: 6307.3872 - val_loss: 167169440.0000 - val_mae: 8989.1465
## Epoch 48/50
##   1/108 [..............................] - ETA: 0s - loss: 139557536.0000 - mae: 8465.5859
108/108 [==============================] -
0s 662us/step - loss: 86066968.0000 - mae: 6315.9707 - val_loss: 167074192.0000 - val_mae: 8992.2041
## Epoch 49/50
##   1/108 [..............................] - ETA: 0s - loss: 72218888.0000 - mae: 5613.7661
105/108 [============================>.] - ET
A: 0s - loss: 86011232.0000 - mae: 6289.5762
108/108 [==============================] - 0s 701us/step - loss: 86031952.0000 - mae: 6296.34
38 - val_loss: 166938976.0000 - val_mae: 9009.5381
## Epoch 50/50
##   1/108 [..............................] - ETA: 0s - loss: 98682816.0000 - mae: 6567.3984
108/108 [==============================] - 0s
675us/step - loss: 85824752.0000 - mae: 6279.8042 - val_loss: 166809728.0000 - val_mae: 9125.3555
```

```
# Plot the training progress (Look for where validation loss stops dropping)
plot(history)
```



```
# 4. Prediction and RMSE
# Get predictions
dnn_preds <- model %>% predict(x_test)
```

```
## 134/134 - 0s - 156ms/epoch - 1ms/step
```

```
# Calculate RMSE
dnn_rmse <- sqrt(mean((dnn_preds - y_test)^2))
print(paste("DNN Test RMSE:", round(dnn_rmse, 2)))
```

```
## [1] "DNN Test RMSE: 10145.82"
```

==

Re-Run After Adjusting For Collinear Predictors

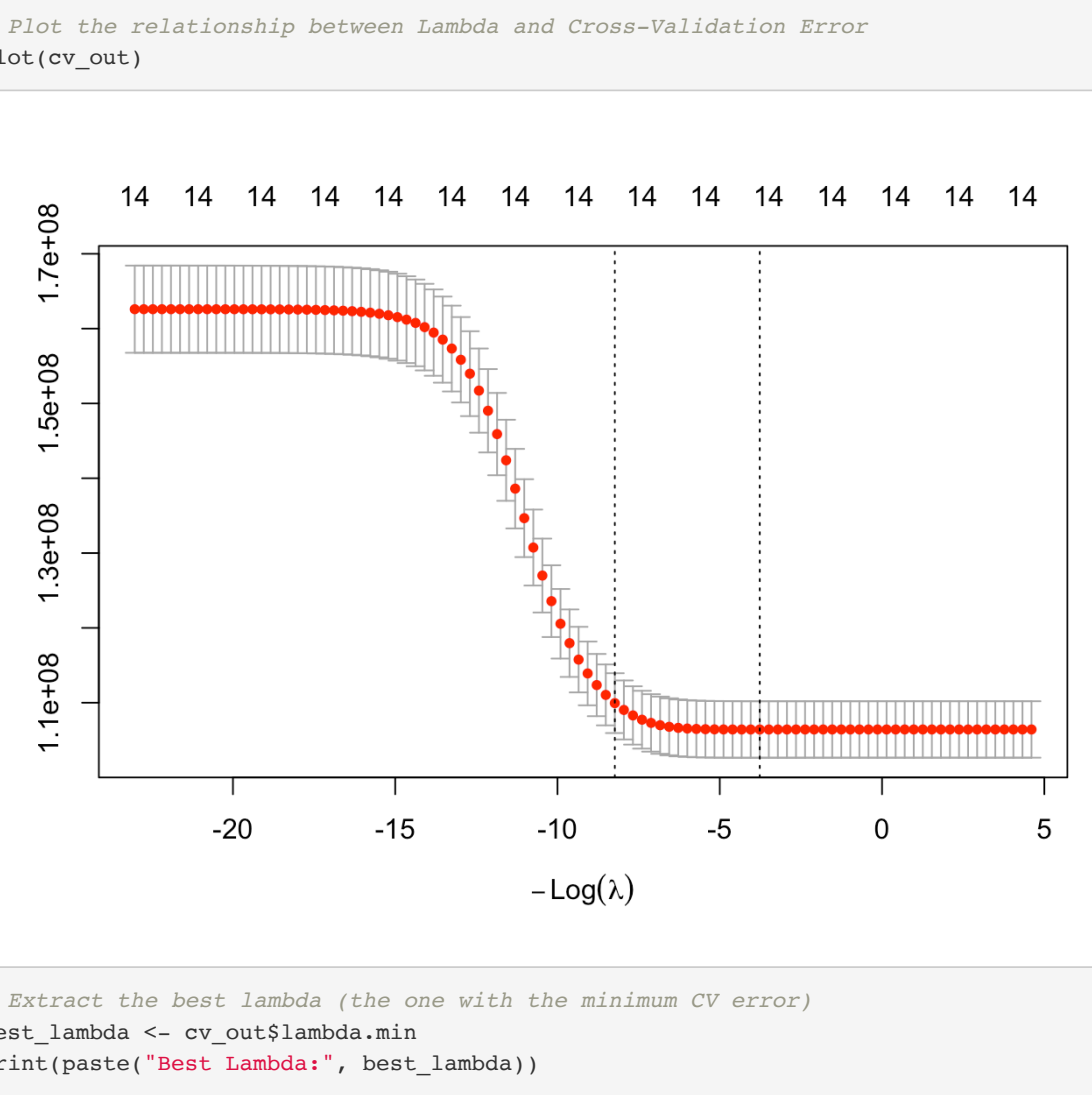
Let's run the three models for this study again, but this time I will be removing redundant variables from the data to solve issue for multicollinearity

```
train_data <- subset(train_data, select = -c(tmin, tmax, tmin01, tdmean, ndvi, vpdmax))
test_data <- subset(test_data, select = -c(tmin, tmax, tmin01, tdmean, ndvi, vpdmax))
```

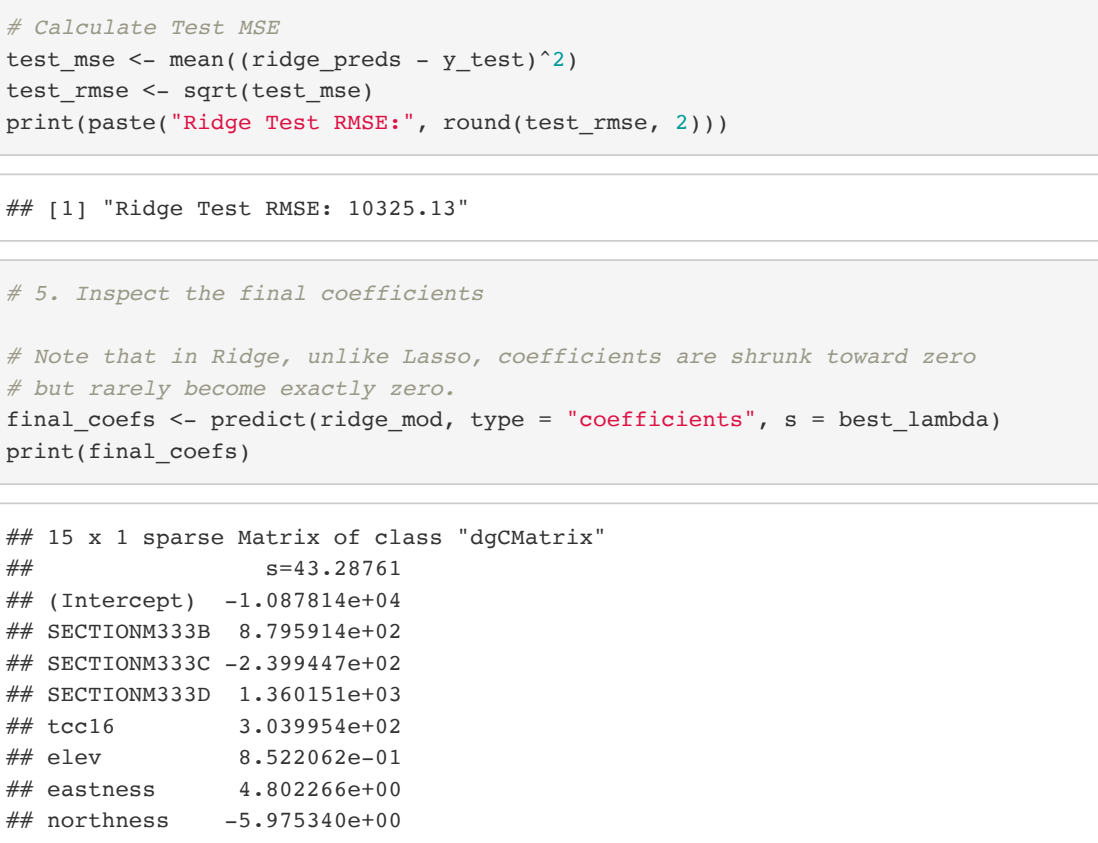
==

Ridge/Lasso Regression

```
# RIDGE REGRESSION

# 1. Data Preparation

# Create numeric matrices for predictors (dropping the Intercept column with [, -1])
# We use the formula of our desired model (e.g., VOL.BF ~ .)
x_train <- model.matrix(VOL.BF ~ ., data = train_data)[, -1]
y_train <- train_data$VOL.BF

x_test <- model.matrix(VOL.BF ~ ., data = test_data)[, -1]
y_test <- test_data$VOL.BF

# Define a grid of lambda values to test (from 10^10 to 10^-2)
grid <- 10^seq(10, -2, length = 100)

# 2. Fit Initial Ridge Model

# Fit ridge regression models for all lambdas in the grid
ridge_mod <- glmnet(x_train, y_train, alpha = 0, lambda = grid, thresh = 1e-12)

# Optional: Visualize how coefficients shrink as lambda increases
plot(ridge_mod, xvar = "lambda", label = TRUE)
```



```
# 3. Use CV to identify most optimal lambda value
set.seed(123) # For reproducibility of the CV folds
cv_out <- cv.glmnet(x_train, y_train, alpha = 0, lambda = grid)

# Plot the relationship between Lambda and Cross-Validation Error
plot(cv_out)
```



```
# Extract the best lambda (the one with the minimum CV error)
best_lambda <- cv_out$lambda.min
print(paste("Best Lambda:", best_lambda))
```

```
## [1] "Best Lambda: 43.2876128108306"
```

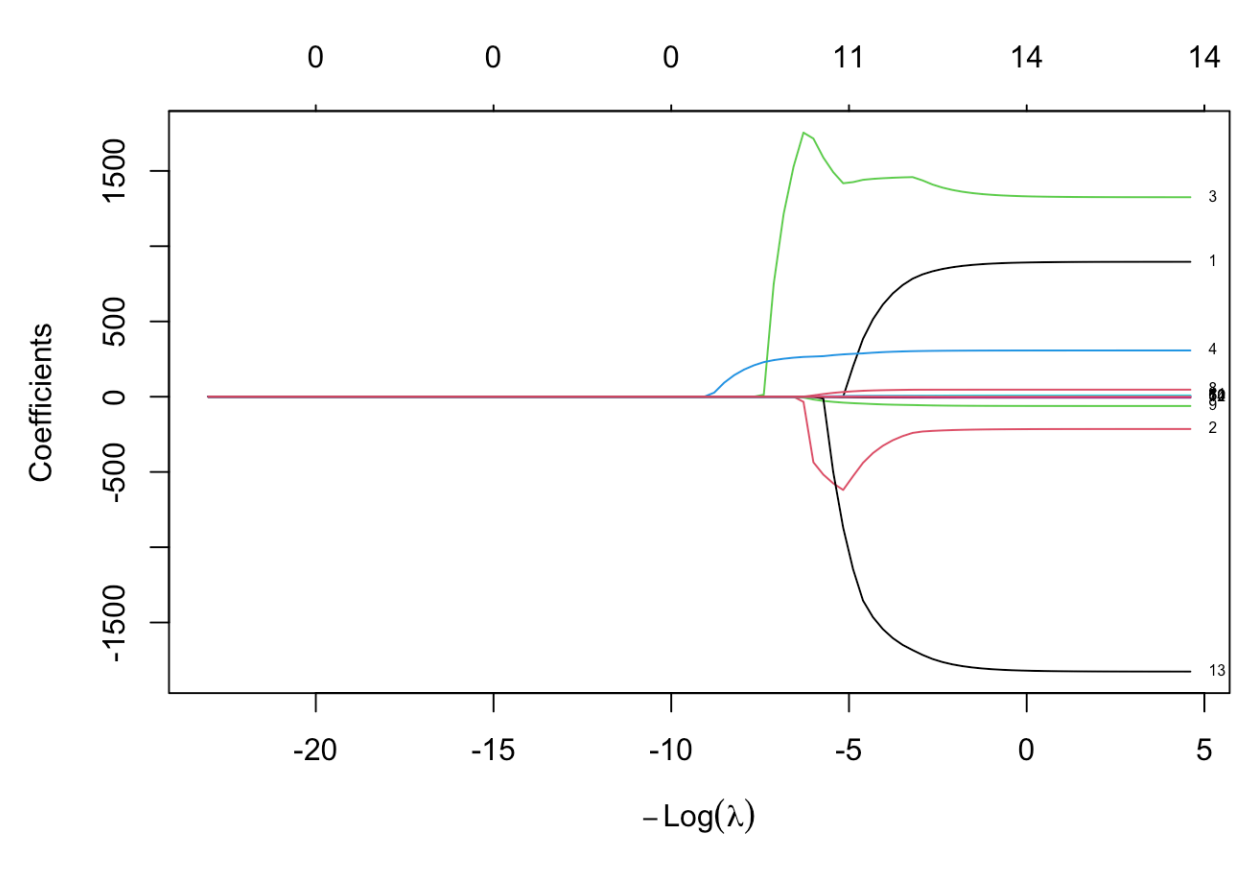
```
# 4. Predict outcomes for the test set using the best lambda
ridge_preds <- predict(ridge_mod, s = best_lambda, newx = x_test)

# Calculate Test MSE
test_mse <- mean((ridge_preds - y_test)^2)
test_rmse <- sqrt(test_mse)
print(paste("Ridge Test RMSE:", round(test_rmse, 2)))
```

```
## [1] "Ridge Test RMSE: 10325.13"
```

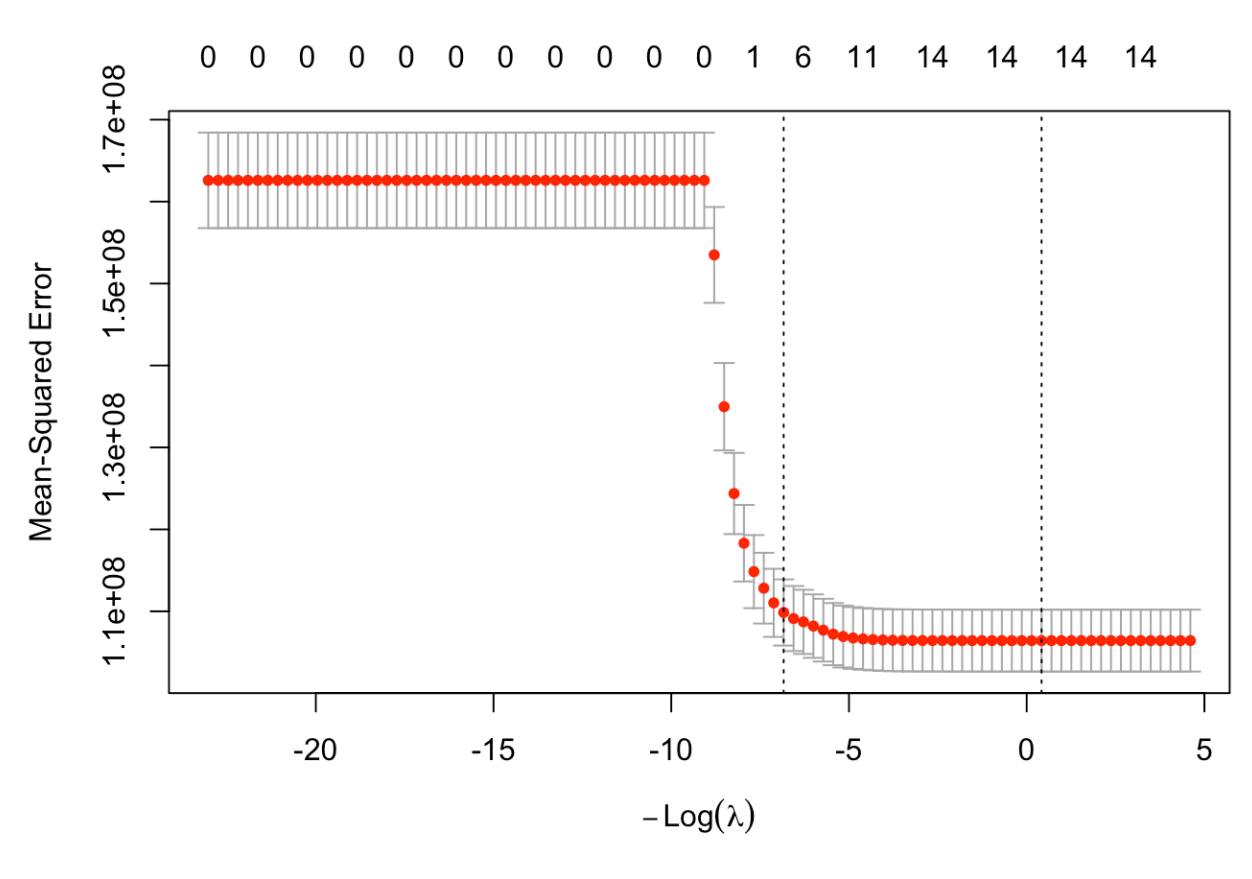
```
# 5. Inspect the final coefficients

# Note that in Ridge, unlike Lasso, coefficients are shrunk toward zero
# but rarely become exactly zero.
final_coefs <- predict(ridge_mod, type = "coefficients", s = best_lambda)
print(final_coefs)
```

```
## 15 x 1 sparse Matrix of class "dgCMatrix"
##                   s=43.28761
## (Intercept)  -1.087814e+04
## SECTIONM333B  8.795914e+02
## SECTIONM333C -2.399447e+02
## SECTIONM333D  1.360151e+03
## tcc16         3.039954e+02
## elev          8.522062e-01
## eastness      4.802266e+00
## northness    -5.975340e+00
## tri           4.701018e+01
## tpi          -6.112714e+01
## ppt           3.810702e+00
## tmean         8.306964e+00
## def          -3.539703e+00
## tnt          -1.775685e+03
## evi          -7.644851e-02
```

```
# LASSO REGRESSION

# 1. Data Preparation

# Create numeric matrices for predictors (dropping the Intercept column with [, -1])
# We use the formula of our desired model (e.g., VOL.BF ~ .)
x_train <- model.matrix(VOL.BF ~ ., data = train_data)[, -1]
y_train <- train_data$VOL.BF

x_test <- model.matrix(VOL.BF ~ ., data = test_data)[, -1]
y_test <- test_data$VOL.BF

# Define a grid of lambda values to test (from 10^10 to 10^-2)
grid <- 10^seq(10, -2, length = 100)

# 2. Fit Initial Lasso Model

# Fit lasso regression models for all lambdas in the grid
lasso_mod <- glmnet(x_train, y_train, alpha = 1, lambda = grid, thresh = 1e-12)

# Optional: Visualize how coefficients shrink as lambda increases
plot(lasso_mod, xvar = "lambda", label = TRUE)
```



```
# 3. Use CV to identify most optimal lambda value
set.seed(123) # For reproducibility of the CV folds
cv_out <- cv.glmnet(x_train, y_train, alpha = 1, lambda = grid)

# Plot the relationship between Lambda and Cross-Validation Error
plot(cv_out)
```



```
# Extract the best lambda (the one with the minimum CV error)
best_lambda <- cv_out$lambda.min
print(paste("Best Lambda:", best_lambda))
```

```
## [1] "Best Lambda: 0.657933224657568"
```

```
# 4. Predict outcomes for the test set using the best lambda
lasso_preds <- predict(lasso_mod, s = best_lambda, newx = x_test)

# Calculate Test MSE
test_mse <- mean((lasso_preds - y_test)^2)
test_rmse <- sqrt(test_mse)
print(paste("Lasso Test RMSE:", round(test_rmse, 2)))
```

```
## [1] "Lasso Test RMSE: 10325.52"
```

```
# 5. Inspect the final coefficients

# Note that in Ridge, unlike Lasso, coefficients are shrunk toward zero
# but rarely become exactly zero.
final_coefs <- predict(lasso_mod, type = "coefficients", s = best_lambda)
print(final_coefs)
```

```
## 15 x 1 sparse Matrix of class "dgCMatrix"
##                s=0.6579332
## (Intercept)  -1.118109e+04
## SECTIONM333B  8.932392e+02
## SECTIONM333C -2.151436e+02
## SECTIONM333D  1.329234e+03
## tcc16         3.071417e+02
## elev          9.557834e-01
## eastness      4.842253e+00
## northness    -6.029404e+00
## tri           4.680359e+01
## tpi          -6.156710e+01
## ppt           3.863075e+00
## tmean         8.569801e+00
## def          -3.482551e+00
## tnt          -1.822133e+03
## evi          -8.485558e-02
```

==

Random Forest

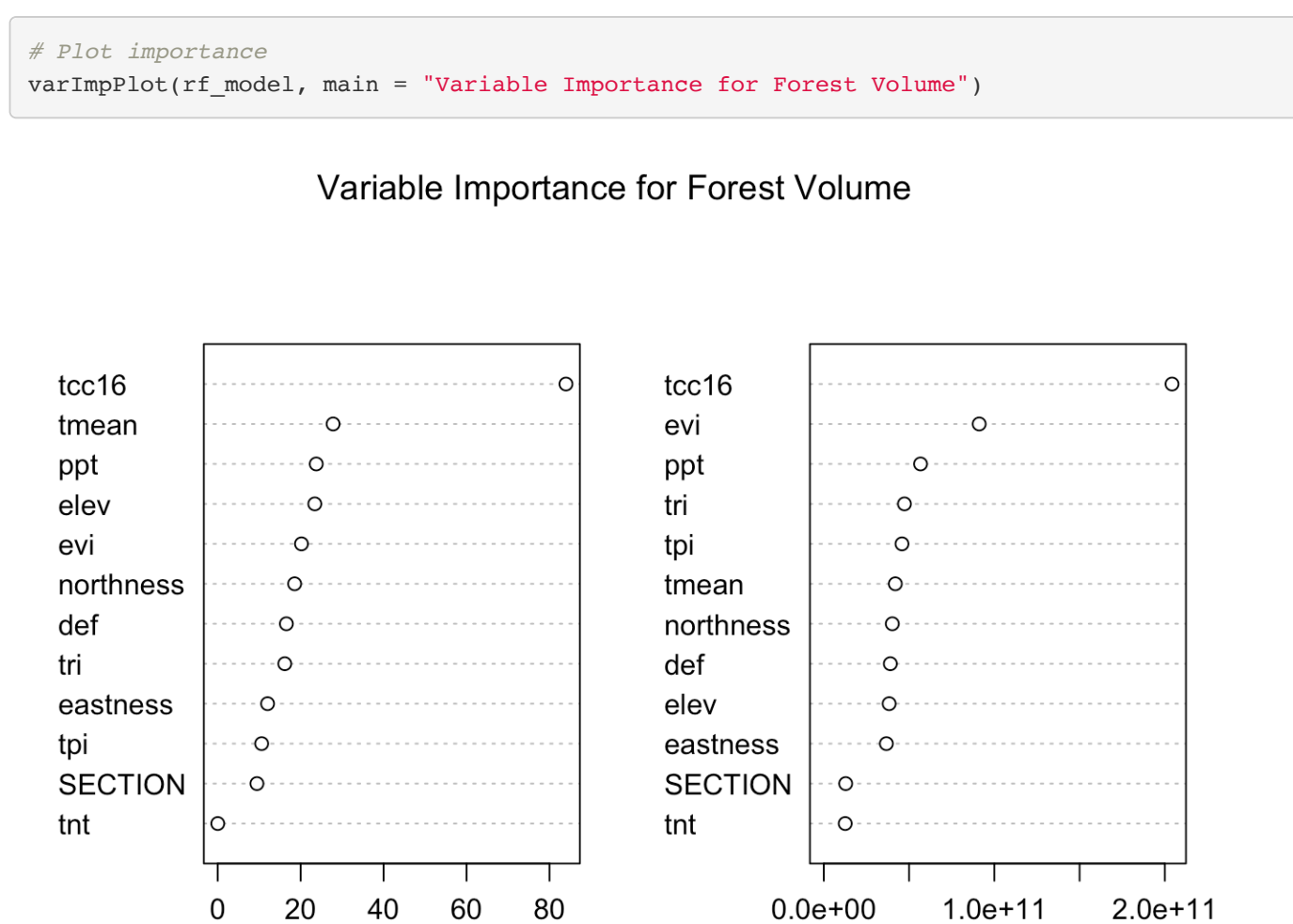
```
# 1. Fit the Random Forest Model

# Set seed for reproducibility because RF uses random sampling
set.seed(123)


# Parameters:
# mtry: Subset of randomly sampled variables to be candidates at each split.
#       For regression, the default is p/3 (p = 18, so mtry = 6)
# ntree: Number of trees to grow. 500 is usually enough.
# importance: Set to TRUE so we can see which predictors matter most.

# Fit the model
rf_model <- randomForest(VOL.BF ~ .,
                         data = train_data,
                         ntree = 500,
                         mtry = 6,
                         importance = TRUE)

# View the basic model output
print(rf_model)
```

```
##
## Call:
##  randomForest(formula = VOL.BF ~ ., data = train_data, ntree = 500,      mtry = 6, importance = TRUE)
##                Type of random forest: regression
##                      Number of trees: 500
## No. of variables tried at each split: 6
##
##           Mean of squared residuals: 102422316
##                     % Var explained: 36.97
```

```
# 2. View variable importance scores
importance(rf_model)
```

```
##              %IncMSE IncNodePurity
## SECTION    9.4563879   12821428458
## tcc16     83.9754037  204210215855
## elev      23.4074526   38310723066
## eastness  12.0233068   36684003703
## northness 18.5443366   40163234304
## tri       16.1671953   47276542683
## tpi       10.5924464   45834858791
## ppt       23.7614009   56714498354
## tmean     27.8236937   41937818863
## def       16.5546488   39004832099
## tnt        0.0119422   12551166365
## evi       20.1956631   91125093573
```

```
# Plot importance
varImpPlot(rf_model, main = "Variable Importance for Forest Volume")
```

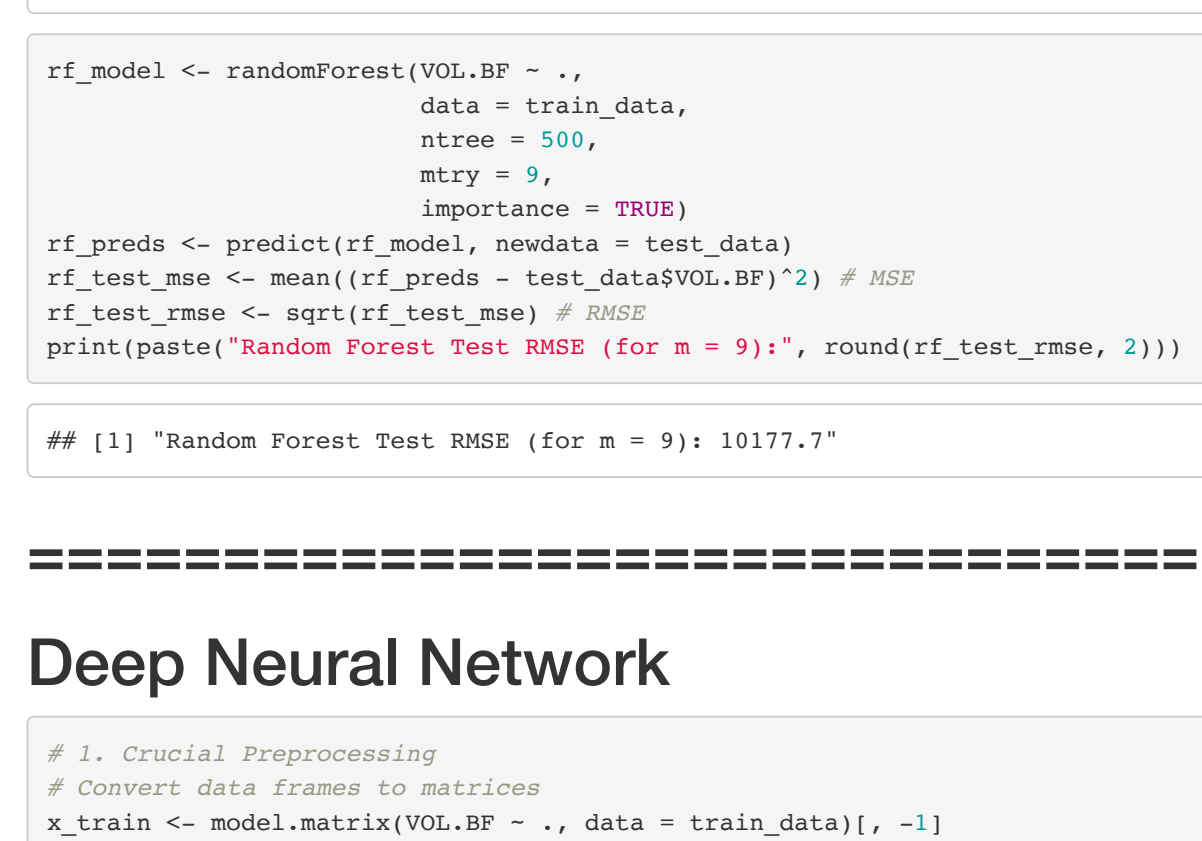


```
# 3. Make Predictions and Calculate Test Error
rf_preds <- predict(rf_model, newdata = test_data)

# Calculate Test Mean Squared Error (MSE)
rf_test_mse <- mean((rf_preds - test_data$VOL.BF)^2)

# Calculate Root Mean Squared Error (RMSE) for interpretability
# (This is the error in the actual units of VOL.BF)
rf_test_rmse <- sqrt(rf_test_mse)

print(paste("Random Forest Test RMSE:", round(rf_test_rmse, 2)))
```

```
## [1] "Random Forest Test RMSE: 10131.6"
```

```
# Tuning the Model using OOB Error
# Find the best mtry value automatically
tuned_mtry <- tuneRF(x = train_data[, -which(names(train_data) == "VOL.BF")],
                     y = train_data$VOL.BF,
                     stepFactor = 1.5,
                     improve = 0.01,
                     ntree = 500)
```

```
## mtry = 4  OOB error = 101980425
## Searching left ...
## mtry = 3     OOB error = 101664555
## 0.003097365 0.01
## Searching right ...
## mtry = 6     OOB error = 102988247
## -0.009882499 0.01
```



```
print("Optimal m = 9")
```

```
## [1] "Optimal m = 9"
```

```
rf_model <- randomForest(VOL.BF ~ .,
                         data = train_data,
                         ntree = 500,
                         mtry = 9,
                         importance = TRUE)
rf_preds <- predict(rf_model, newdata = test_data)
rf_test_mse <- mean((rf_preds - test_data$VOL.BF)^2) # MSE
rf_test_rmse <- sqrt(rf_test_mse) # RMSE
print(paste("Random Forest Test RMSE (for m = 9):", round(rf_test_rmse, 2)))
```

```
## [1] "Random Forest Test RMSE (for m = 9): 10177.7"
```

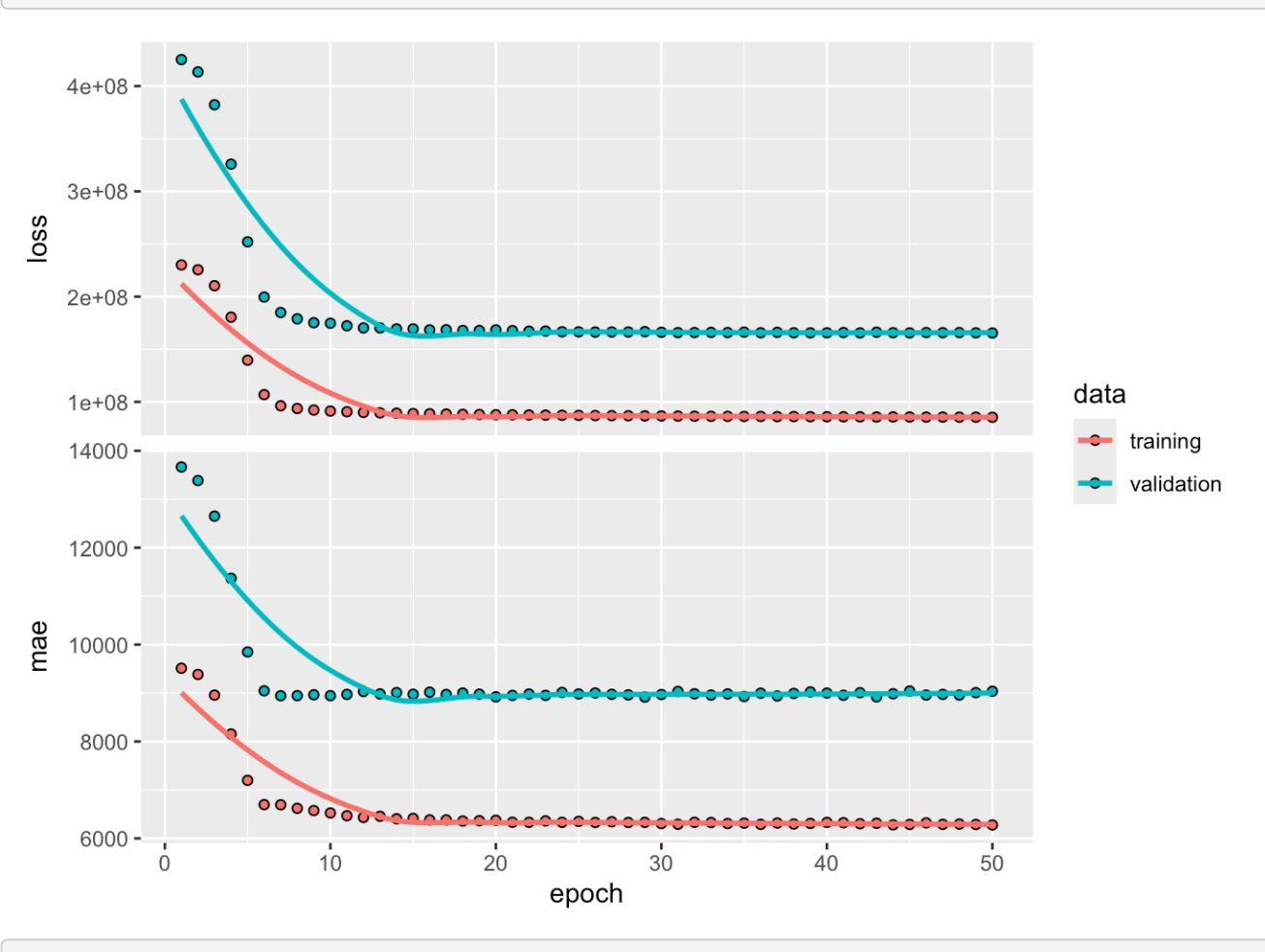
==

Deep Neural Network

```
# 1. Crucial Preprocessing
# Convert data frames to matrices
x_train <- model.matrix(VOL.BF ~ ., data = train_data)[, -1]
y_train <- train_data$VOL.BF
x_test <- model.matrix(VOL.BF ~ ., data = test_data)[, -1]
y_test <- test_data$VOL.BF

# Feature Scaling (Standardization)
# We calculate mean/sd from training and apply to both sets
mean <- colMeans(x_train) # calculating means of each column from x_train matrix
std <- apply(x_train, 2, sd) # calculating sd for each column (2 tells apply() to work with columns)

# Scaling the training variables
x_train <- scale(x_train, center = mean, scale = std)
x_test <- scale(x_test, center = mean, scale = std)

# 2. Define Architecture
dnn_model <- keras_model_sequential() %>%
  # Layer 1: Input layer (must match your 18 predictors)
  layer_dense(units = 64, activation = "relu", input_shape = ncol(x_train)) %>%
  # Layer 2: Hidden layer
  layer_dense(units = 32, activation = "relu") %>%
  # Layer 3: Hidden layer
  layer_dense(units = 16, activation = "relu") %>%
  # Output layer: 1 unit for Regression (no activation function for raw numeric output)
  layer_dense(units = 1)

# Compile the model
dnn_model %>% compile(
  optimizer = "rmsprop",
  loss = "mse",      # We optimize for Mean Squared Error
  metrics = c("mae") # We track Mean Absolute Error (easier to read)
)

# 3. Training the Model
history <- dnn_model %>% fit(
  x_train, y_train,
  epochs = 50,
  batch_size = 32,
  validation_split = 0.2, # Automatically uses 20% of training data for "OOB-like" checks
  verbose = 1
)
```

```
## Epoch 1/50
##   1/108 [..............................] - ETA: 41s - loss: 220296240.0000 - mae: 8905.1289□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ 86/108 [=====================>.......]
- ETA: 0s - loss: 238595696.0000 - mae: 9722.4941 □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 1s 2ms/step - loss: 230102624.0000 - mae: 9
512.0391 - val_loss: 425157120.0000 - val_mae: 13666.0684
## Epoch 2/50
##   1/108 [..............................] - ETA: 0s - loss: 123224640.0000 - mae: 7042.7314□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] -
0s 654us/step - loss: 225554272.0000 - mae: 9383.9873 - val_loss: 413454176.0000 - val_mae: 13386.3438
## Epoch 3/50
##   1/108 [..............................] - ETA: 0s - loss: 156578208.0000 - mae: 7333.9155□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] -
0s 655us/step - loss: 210366416.0000 - mae: 8957.0996 - val_loss: 382258848.0000 - val_mae: 12651.4170
## Epoch 4/50
##   1/108 [..............................] - ETA: 0s - loss: 341335104.0000 - mae: 11287.3887□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================]
- 0s 644us/step - loss: 180480160.0000 - mae: 8154.5562 - val_loss: 325848320.0000 - val_mae: 11367.6914
## Epoch 5/50
##   1/108 [..............................] - ETA: 0s - loss: 124230416.0000 - mae: 8152.6318□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] -
0s 639us/step - loss: 139600144.0000 - mae: 7198.6348 - val_loss: 252015168.0000 - val_mae: 9848.4795
## Epoch 6/50
##   1/108 [..............................] - ETA: 0s - loss: 164987792.0000 - mae: 8404.9219□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] -
0s 655us/step - loss: 106862152.0000 - mae: 6697.0811 - val_loss: 199575584.0000 - val_mae: 9045.6406
## Epoch 7/50
##   1/108 [..............................] - ETA: 0s - loss: 125955128.0000 - mae: 6891.8608□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] -
0s 680us/step - loss: 96276688.0000 - mae: 6693.8589 - val_loss: 184890544.0000 - val_mae: 8943.8740
## Epoch 8/50
##   1/108 [..............................] - ETA: 0s - loss: 107259816.0000 - mae: 8097.8008□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] -
0s 671us/step - loss: 93719848.0000 - mae: 6620.7236 - val_loss: 178884944.0000 - val_mae: 8946.2930
## Epoch 9/50
##   1/108 [..............................] - ETA: 0s - loss: 126575600.0000 - mae: 7670.8867□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□106/108 [============================>.] -
ETA: 0s - loss: 92592464.0000 - mae: 6581.6172 □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s 706us/step - loss: 92246288.0000 - mae: 657
4.3555 - val_loss: 175211216.0000 - val_mae: 8964.0234
## Epoch 10/50
##   1/108 [..............................] - ETA: 0s - loss: 154233952.0000 - mae: 8658.4277□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□105/108 [============================>.] -
ETA: 0s - loss: 90876960.0000 - mae: 6507.8257 □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s 701us/step - loss: 91302232.0000 - mae: 652
4.9810 - val_loss: 174702128.0000 - val_mae: 8945.9854
## Epoch 11/50
##   1/108 [..............................] - ETA: 0s - loss: 59902976.0000 - mae: 5227.2104□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s
649us/step - loss: 90647280.0000 - mae: 6468.2266 - val_loss: 172311984.0000 - val_mae: 8972.3809
## Epoch 12/50
##   1/108 [..............................] - ETA: 0s - loss: 139530304.0000 - mae: 8481.1016□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] -
0s 703us/step - loss: 90001544.0000 - mae: 6431.5850 - val_loss: 170264032.0000 - val_mae: 9031.4141
## Epoch 13/50
##   1/108 [..............................] - ETA: 0s - loss: 105915392.0000 - mae: 7182.8442□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□107/108 [============================>.] -
ETA: 0s - loss: 89624712.0000 - mae: 6456.9355 □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s 696us/step - loss: 89585368.0000 - mae: 645
5.7749 - val_loss: 170253088.0000 - val_mae: 8981.9951
## Epoch 14/50
##   1/108 [..............................] - ETA: 0s - loss: 158173440.0000 - mae: 7086.9678□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] -
0s 668us/step - loss: 89258088.0000 - mae: 6405.4438 - val_loss: 169314304.0000 - val_mae: 9008.9102
## Epoch 15/50
##   1/108 [..............................] - ETA: 0s - loss: 48051360.0000 - mae: 5380.3721□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s
677us/step - loss: 88908208.0000 - mae: 6412.3125 - val_loss: 169365136.0000 - val_mae: 8975.0234
## Epoch 16/50
##   1/108 [..............................] - ETA: 0s - loss: 85026984.0000 - mae: 5848.4824□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s
649us/step - loss: 88664992.0000 - mae: 6381.8281 - val_loss: 168416656.0000 - val_mae: 9017.5986
## Epoch 17/50
##   1/108 [..............................] - ETA: 0s - loss: 55016168.0000 - mae: 5358.3032□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s
674us/step - loss: 88425392.0000 - mae: 6381.5483 - val_loss: 168544912.0000 - val_mae: 8970.9072
## Epoch 18/50
##   1/108 [..............................] - ETA: 0s - loss: 135706304.0000 - mae: 8142.5088□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] -
0s 660us/step - loss: 88195888.0000 - mae: 6359.1201 - val_loss: 167871680.0000 - val_mae: 8999.2578
## Epoch 19/50
##   1/108 [..............................] - ETA: 0s - loss: 58511764.0000 - mae: 5023.8818□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s
677us/step - loss: 88019176.0000 - mae: 6366.0229 - val_loss: 167795440.0000 - val_mae: 8976.4697
## Epoch 20/50
##   1/108 [..............................] - ETA: 0s - loss: 89009584.0000 - mae: 6482.2769□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s
667us/step - loss: 87799840.0000 - mae: 6373.8237 - val_loss: 168406208.0000 - val_mae: 8920.5430
## Epoch 21/50
##   1/108 [..............................] - ETA: 0s - loss: 102221888.0000 - mae: 7416.5625□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□107/108 [============================>.] -
ETA: 0s - loss: 87840176.0000 - mae: 6337.0244 □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s 706us/step - loss: 87753776.0000 - mae: 633
3.4243 - val_loss: 167682816.0000 - val_mae: 8951.4600
## Epoch 22/50
##   1/108 [..............................] - ETA: 0s - loss: 51664604.0000 - mae: 5801.1104□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s
660us/step - loss: 87552064.0000 - mae: 6332.2158 - val_loss: 167165088.0000 - val_mae: 8979.3691
## Epoch 23/50
##   1/108 [..............................] - ETA: 0s - loss: 50225536.0000 - mae: 5250.4590□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s
665us/step - loss: 87403680.0000 - mae: 6361.9229 - val_loss: 167309024.0000 - val_mae: 8952.0664
## Epoch 24/50
##   1/108 [..............................] - ETA: 0s - loss: 121151408.0000 - mae: 7269.8423□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] -
0s 682us/step - loss: 87294392.0000 - mae: 6331.8271 - val_loss: 166606304.0000 - val_mae: 9012.2324
## Epoch 25/50
##   1/108 [..............................] - ETA: 0s - loss: 148165808.0000 - mae: 8278.4062□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] -
0s 669us/step - loss: 87175672.0000 - mae: 6351.6138 - val_loss: 166543792.0000 - val_mae: 8981.3330
## Epoch 26/50
##   1/108 [..............................] - ETA: 0s - loss: 47711380.0000 - mae: 5215.7983□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s
662us/step - loss: 87028344.0000 - mae: 6329.0127 - val_loss: 166316656.0000 - val_mae: 9001.3447
## Epoch 27/50
##   1/108 [..............................] - ETA: 0s - loss: 111355912.0000 - mae: 7304.9541□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□100/108 [==========================>...] -
ETA: 0s - loss: 87842312.0000 - mae: 6384.6680 □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s 722us/step - loss: 86905976.0000 - mae: 634
4.1050 - val_loss: 166404624.0000 - val_mae: 8974.0020
## Epoch 28/50
##   1/108 [..............................] - ETA: 0s - loss: 103064264.0000 - mae: 6735.1675□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] -
0s 659us/step - loss: 86816640.0000 - mae: 6328.4282 - val_loss: 166393152.0000 - val_mae: 8961.8301
## Epoch 29/50
##   1/108 [..............................] - ETA: 0s - loss: 95943704.0000 - mae: 6177.9375□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - ET
A: 0s - loss: 86716208.0000 - mae: 6331.1313□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s 684us/step - loss: 86716208.0000 - mae: 6331.13
13 - val_loss: 166736544.0000 - val_mae: 8916.5781
## Epoch 30/50
##   1/108 [..............................] - ETA: 0s - loss: 67717792.0000 - mae: 5948.3770□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s
659us/step - loss: 86638464.0000 - mae: 6305.9492 - val_loss: 166131168.0000 - val_mae: 8969.7041
## Epoch 31/50
##   1/108 [..............................] - ETA: 0s - loss: 54708680.0000 - mae: 4970.7158□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s
661us/step - loss: 86508360.0000 - mae: 6292.4961 - val_loss: 165805392.0000 - val_mae: 9032.5850
## Epoch 32/50
##   1/108 [..............................] - ETA: 0s - loss: 111777864.0000 - mae: 7650.9805□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] -
0s 653us/step - loss: 86391216.0000 - mae: 6332.7437 - val_loss: 165885696.0000 - val_mae: 8984.2607
## Epoch 33/50
##   1/108 [..............................] - ETA: 0s - loss: 83848880.0000 - mae: 6973.4438□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s
659us/step - loss: 86311288.0000 - mae: 6327.9854 - val_loss: 166039504.0000 - val_mae: 8958.2334
## Epoch 34/50
##   1/108 [..............................] - ETA: 0s - loss: 49000064.0000 - mae: 5278.0679□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s
649us/step - loss: 86251208.0000 - mae: 6306.4644 - val_loss: 165872112.0000 - val_mae: 8981.8438
## Epoch 35/50
##   1/108 [..............................] - ETA: 0s - loss: 112922256.0000 - mae: 6637.1558□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] -
0s 664us/step - loss: 86196952.0000 - mae: 6314.9873 - val_loss: 166309744.0000 - val_mae: 8928.5098
## Epoch 36/50
##   1/108 [..............................] - ETA: 0s - loss: 132318376.0000 - mae: 6974.0566□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] -
0s 656us/step - loss: 86199160.0000 - mae: 6289.4634 - val_loss: 165681984.0000 - val_mae: 8997.2686
## Epoch 37/50
##   1/108 [..............................] - ETA: 0s - loss: 199960736.0000 - mae: 8413.6953□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□107/108 [============================>.] -
ETA: 0s - loss: 86077136.0000 - mae: 6319.6582 □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s 772us/step - loss: 86000832.0000 - mae: 631
6.6187 - val_loss: 166066384.0000 - val_mae: 8939.6025
## Epoch 38/50
##   1/108 [..............................] - ETA: 0s - loss: 170343008.0000 - mae: 8876.3047□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□104/108 [===========================>..] -
ETA: 0s - loss: 85690072.0000 - mae: 6285.8730 □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s 720us/step - loss: 86067656.0000 - mae: 629
6.8584 - val_loss: 165664480.0000 - val_mae: 8992.0732
## Epoch 39/50
##   1/108 [..............................] - ETA: 0s - loss: 105983088.0000 - mae: 7395.2622□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] -
ETA: 0s - loss: 85929144.0000 - mae: 6309.2646 □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s 684us/step - loss: 85929144.0000 - mae: 630
9.2646 - val_loss: 165597952.0000 - val_mae: 9023.0879
## Epoch 40/50
##   1/108 [..............................] - ETA: 0s - loss: 77127264.0000 - mae: 6814.2246□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s
680us/step - loss: 85913144.0000 - mae: 6329.6177 - val_loss: 165633536.0000 - val_mae: 8999.5742
## Epoch 41/50
##   1/108 [..............................] - ETA: 0s - loss: 70350368.0000 - mae: 5958.3223□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - ET
A: 0s - loss: 85859264.0000 - mae: 6323.7539□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s 691us/step - loss: 85859264.0000 - mae: 6323.75
39 - val_loss: 165898176.0000 - val_mae: 8954.0518
## Epoch 42/50
##   1/108 [..............................] - ETA: 0s - loss: 55081532.0000 - mae: 5038.8809□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s
672us/step - loss: 85806752.0000 - mae: 6302.3770 - val_loss: 165543232.0000 - val_mae: 9006.8682
## Epoch 43/50
##   1/108 [..............................] - ETA: 0s - loss: 49803380.0000 - mae: 5166.9565□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s
658us/step - loss: 85645824.0000 - mae: 6313.5000 - val_loss: 166207520.0000 - val_mae: 8920.5439
## Epoch 44/50
##   1/108 [..............................] - ETA: 0s - loss: 165087072.0000 - mae: 7213.6484□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] -
0s 661us/step - loss: 85721024.0000 - mae: 6279.5498 - val_loss: 165720112.0000 - val_mae: 8984.9531
## Epoch 45/50
##   1/108 [..............................] - ETA: 0s - loss: 105679088.0000 - mae: 6317.7812□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] -
0s 667us/step - loss: 85626984.0000 - mae: 6287.6206 - val_loss: 165533744.0000 - val_mae: 9041.5752
## Epoch 46/50
##   1/108 [..............................] - ETA: 0s - loss: 89334960.0000 - mae: 7214.4263□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s
658us/step - loss: 85541880.0000 - mae: 6319.9067 - val_loss: 165868160.0000 - val_mae: 8958.9043
## Epoch 47/50
##   1/108 [..............................] - ETA: 0s - loss: 32095552.0000 - mae: 4043.8665□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s
653us/step - loss: 85587056.0000 - mae: 6289.4170 - val_loss: 165742624.0000 - val_mae: 8972.5371
## Epoch 48/50
##   1/108 [..............................] - ETA: 0s - loss: 55016584.0000 - mae: 5642.4219□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] - 0s
648us/step - loss: 85473208.0000 - mae: 6295.7158 - val_loss: 165855744.0000 - val_mae: 8958.2363
## Epoch 49/50
##   1/108 [..............................] - ETA: 0s - loss: 127642856.0000 - mae: 8395.0029□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] -
0s 672us/step - loss: 85479824.0000 - mae: 6287.1196 - val_loss: 165622000.0000 - val_mae: 9008.0977
## Epoch 50/50
##   1/108 [..............................] - ETA: 0s - loss: 110459032.0000 - mae: 7232.6729□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□108/108 [==============================] -
0s 673us/step - loss: 85386688.0000 - mae: 6277.0469 - val_loss: 165461520.0000 - val_mae: 9036.4414
```

```
# Plot the training progress (Look for where validation loss stops dropping)
plot(history)
```



```
# 4. Prediction and RMSE
# Get predictions
dnn_preds <- dnn_model %>% predict(x_test)
```

```
## 134/134 - 0s - 113ms/epoch - 845us/step
```

```
# Calculate RMSE
dnn_rmse <- sqrt(mean((dnn_preds - y_test)^2))
print(paste("DNN Test RMSE:", round(dnn_rmse, 2)))
```

```
## [1] "DNN Test RMSE: 10102.84"
```

==

DNN provided the lowest MSE/RMSE

```
range_VOL.BF <- max(test_data$VOL.BF) - min(test_data$VOL.BF)
range_VOL.BF
```

```
## [1] 85885.81
```

```
dnn_rmse/range_VOL.BF
```

```
## [1] 0.1176311
```

DNN provides a RMSE of 10090.94, which is a model prediction error that is about 11.75% of the full range of the response variable VOL.BF.

The simple baseline intercept-only linear regression model performed to score a RMSE 12802.27, significantly higher than that of DNN.

DNN performed significantly better than the baseline intercept-only regression model.